

虚拟细胞-解题思路

一、项目概述

1.1 项目名称

基于条件响应预测的酿酒酵母虚拟细胞建模

1.2 参赛方向

- 赛道：世界人工智能开源大赛（GOAI）· AI for Research 前沿探索赛道
- 赛道类型：算法赛（提供数据集与评估框架，提交可运行、可复现的解决方案）
- 赛题方向：虚拟细胞（Virtual Cell）—— 基于酵母扰动蛋白质组数据的条件响应预测
- 发起方：Datawhale、上海人工智能实验室、西湖大学、浙江大学联合发起

1.3 方案概述

本方案以赛题推荐的建模思路为框架基础，构建一个「条件 → 蛋白质组向量」的回归模型，并以渐进式路线从基线稳步推进到具备零样本泛化能力的最终模型。

赛题推荐框架：把菌株、化合物、实验条件分别编码 → 模型学习它们的交互 → 输出蛋白质组预测（同时考虑测量仪器的偏差）。本任务是 OOD（Out-of-Distribution，分布外泛化，预测训练时未见过的实体）任务，直接从条件生成响应，**不需要输入 control（对照样本，代表"无扰动"的细胞状态，是所有 Δ 计算的参照基准）**。

维度	方案选择	核心保障
研究对象	酿酒酵母扰动蛋白质组	—
科学问题	AI 能否跨细胞背景预测遗传/化学扰动后的蛋白组响应？	—
Baseline 架构	条件编码 → MLP → mask-aware 输出（三步极简）	可行性：仅用成熟组件，单机可跑
提分架构	推荐框架完整实现 + 残差分解 + 外部表征 + 蛋白先验	创新性：六大创新点（见第五章）
稳定性保障	mask-aware 全流程、仅训练集统计量、正交划分、与基线对比	杜绝信息泄漏与虚假高分
准确性保障	分桶报告、多指标联合、embedding 消融	提升可归因，非指标欺骗
方法路线	基线跑通 → 条件编码 MLP → 框架完整实现 → 实体表征增强 → 科学意义	每步可验证、可回退

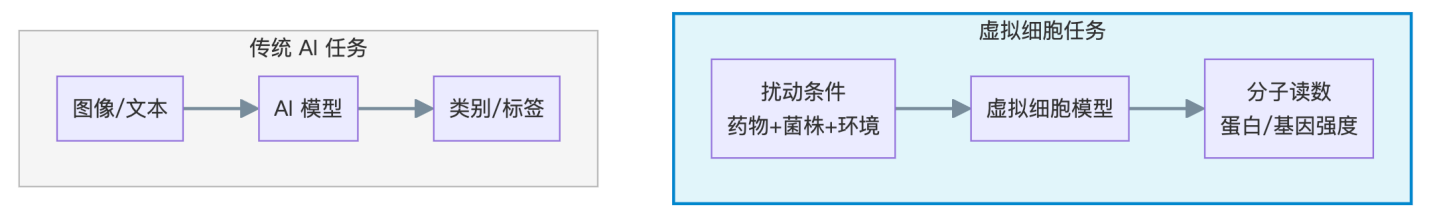
二、科学问题理解

2.1 科学问题与研究对象

2.1.1 什么是虚拟细胞（AI Virtual Cell, AIVC）

虚拟细胞的目标是**用计算机近似细胞对外界扰动的响应**。与图像/文本任务不同，其输出不是类别，而是一组具有明确生物意义的分子读数。

一图看懂虚拟细胞：



简单理解，虚拟细胞就是一个「细胞模拟器」：

- **输入**：你给细胞施加什么条件（加什么药、用什么菌株、什么温度）
- **输出**：细胞内部的蛋白质会发生什么变化（4,232 个蛋白的强度值）
- **目标**：不用真的做实验，就能预测结果

和传统 AI 任务的区别：图像分类输出一个标签（猫/狗），虚拟细胞输出一组长达 4,232 个数值的「细胞状态画像」。

2.1.2 概念演进脉络

虚拟细胞的发展可以简单理解为「从手工方程到 AI 学习」的转变：

阶段	想法	问题
早期（2012）	科学家手写方程描述细胞每个子模块怎么运作	细胞太复杂，方程写不全，只能模拟几百个基因的小细胞
中期（2024）	提出「AI 虚拟细胞」：让 AI 从数据中自己学细胞规律	当时只是概念，还没有大规模实验验证
现在（本赛题）	用 AI 预测酵母在药物/基因扰动下的蛋白质组变化	就是你要解决的问题

一句话理解：以前科学家试图「写方程描述细胞」，现在我们试图「让 AI 从实验数据中学出细胞规律」。本赛题是这个方向的首次大规模实验验证。

2.1.3 为什么选择酵母（研究对象）

赛题选择**酿酒酵母**（做啤酒/面包用的那种酵母）作为研究对象，原因很朴素：

原因	解释
足够复杂但可控	酵母是真核细胞（和人类细胞结构类似），但只有一个物种，实验条件可以系统控制
长得快、便宜	几小时就能繁殖一代，培养成本极低，可以大量做实验
蛋白组更贴近功能	赛题测的是蛋白质（真正干活的分子），不是 mRNA（只是图纸），更接近细胞真实状态
无伦理限制	不像人源细胞有伦理审查，验证周期短

一句话理解：酵母就像生物学里的「果蝇」——简单、好养、够用，是研究真核细胞的经典模式生物。

2.1.4 本赛题的核心科学问题

现在的 AI 能否跨细胞背景预测遗传扰动（或化学扰动）后的蛋白组响应？

本赛题变量很多：扰动条件为 **5 维组合**——不同酵母菌株 × 不同化学扰动 × 不同培养基 × 不同温度 × 时间序列（6 菌株 × **56 化合物（含对照，其中 55 个为化学扰动）** × 2 培养基 × 2 温度 × 6 时间点）。任务并非在已知组合上做插值，而是检验模型能否将训练数据中学到的生物规律**迁移到新化合物、新菌株以及二者同时未知的场景**。

问题的难度边界（量化解）：

- 训练集（split_final=train）：共 5,920 个样本，包含处理样本、对照样本（DMSO/Water）与质控样本；涉及 4 菌株 × 38 化合物
- 泛化目标：未见菌株 × 未见化合物 × 时间外推，**最极端场景仅 269 个样本**
- 输出空间：5,243 维蛋白质组，其中约 20% 蛋白在 ≥80% 样本中缺失（需过滤至 4,232 维）
- 核心挑战：**小样本下的高维外推**，模型必须利用外部知识（基因组、化学结构）弥补数据稀疏

2.2 科学意义

2.2.1 对科学领域的意义

- 应用价值**：帮助研究者在开展实验前筛选更有价值的「菌株—化合物」组合，缩小需实际测量的范围，加速假说验证与药物/菌株筛选（如预测某菌株的酿酒能力或抗逆性）。
- 方法价值**：任务同时包含**高维输出、缺失观测、批次差异、实体级零样本泛化**，是比常规随机切分更接近真实科研使用场景的问题，对 AI4Science 范式有示范意义。如果在此数据集上取得突破，方法论可直接迁移到人类细胞系、药物筛选等更复杂场景。
- 闭环意义**：虚拟细胞可使 AI 与科学直觉、数据、实验验证形成更高效的发现闭环，而不只是辅助分析工具。

2.2.2 三大数据支柱与闭环系统

类比理解：建设虚拟细胞就像培养一个「细胞生物学研究生」，需要三样东西：

支柱	类比	本赛题对应	作用
先验知识	教科书和文献	菌株基因组、化合物化学结构、蛋白通路信息	提供背景知识，帮助模型理解「新实体」（5.3 节）
静态状态	实验课观察	蛋白质组的静态强度数据	知道正常状态下各蛋白是多少
动态响应	做扰动实验	药物/基因扰动后的蛋白组变化	本赛题的核心建模目标

一句话理解：就像研究生读文献（先验知识）、观察正常细胞（静态状态）、做扰动实验（动态响应）。本赛题聚焦第三个环节——**预测扰动后的响应**。长期目标可进一步形成闭环（模型建议下一个最有价值的实验，见 5.5.2）。

三、技术方案与预期方法路线

3.1 技术方案

3.1.1 任务形式化

每个样本由菌株、化合物、培养基、温度、处理时间及测量上下文共同描述，模型输出该样本对应的完整蛋白质组强度向量：

代码块

1

输入：菌株 + 化合物 + 实验条件（培养条件上下文）

2

输出：5,243 维蛋白质组强度向量（→ 过滤后 4,232 维）

这是一个 **OOD 条件生成/回归任务**：直接从条件生成响应，**不需要输入 control 样本**。这一设计保证了模型学到的是跨条件的泛化规律，而非简单记住 control 与处理之间的映射。输出是高维连续向量（4,232 个蛋白的强度值），更接近「生成」任务而非「分类」任务。

3.1.1a 模拟数据样例（帮助你直观理解数据长什么样）

以下为**模拟数据**（非真实赛题数据），用于帮助理解数据格式。真实数据以官网发布为准。

元数据样例（metadata CSV，前 5 行 × 关键字段）：

sample_ID	strain	chemical	medium	temperature	time	data_source	instrument
S00001	CRD	drug_A	YPD	30C	2h	WAYB	timsTOF
S00002	CRD	drug_A	YPD	30C	6h	WAYB	timsTOF
S00003	CRD	DMSO	YPD	30C	2h	WAYB	timsTOF
S00004	DHY210	drug_B	SD	37C	4h	WAYC	QExactive
S00005	SK1	drug_C	YPD	30C	8h	WAYB	timsTOF

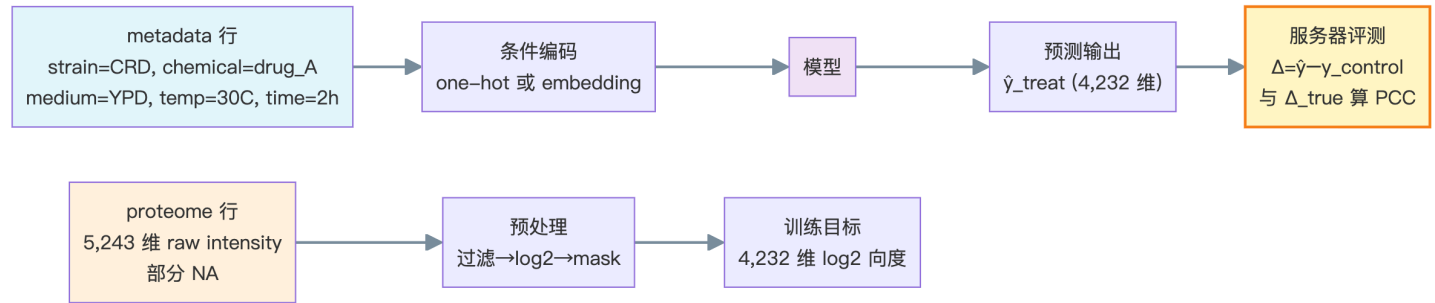
读法：每行一个样本。S00003 是 drug_A 的溶剂对照（DMSO），与 S00001 同条件（CRD/YPD/30C/2h/timsTOF/P01），用于计算 $\Delta = y_{\text{treat}} - y_{\text{control}}$ 。S00005 是质控样本，不参与建模但可用于批次校正。

蛋白质组数据样例（proteome CSV，前 3 行 × 前 6 蛋白）：

sample_ID	protein_1	protein_2	protein_3	protein_4	protein_5	...
S00001	125000.3	89200.1	NA	456000.7	12300.5	...
S00002	118000.5	91000.3	23400.8	441000.2	NA	...
S00003	130000	87000.5	NA	462000.1	13100	...

读法：值为 raw intensity（原始质谱强度），数值范围通常 $10^3 \sim 10^6$ 。NA 表示该蛋白在该样本中未检测到（不是 0）。log2 转换后值约 10~20。

一条样本的完整数据流：



3.1.2 数据规模与组成

数据文件清单（文件名来自赛题解读报告）：

数据文件名	样本数	主要内容	开放情况
WAYB_WAYC_metadata_train_val.csv	8,958	15 个条件与测量字段	开放
WAYB_WAYC_proteome_raw_train_val.csv	8,958	5,243 个蛋白的 raw intensity	开放
WAYB_WAYC_metadata_test.csv	4,454	15 个条件与测量字段	开放
WAYB_WAYC_proteome_raw_test.csv	4,454	仅用于主办方内部评测	不开放

数据维度构成：6 菌株 × 56 化合物 × 2 培养基 × 2 温度 × 6 时间点（+ 质控样本）。化合物维度最多，**化合物响应预测是主要任务。**

关于化合物数量：数据维度为 56（含对照），正交划分为 55（38 训练 + 6 验证 + 11 测试）。56 是数据集化合物维度，55 是参与正交划分的化合物数。

实体划分（精确数字）：

实体	训练	验证	测试	合计
菌株	4	1	1	6
化合物	38	6	11	55

元数据 15 个字段分类（两类，不可混用）：

类别	字段（已确认）	角色	建模方式
生物条件	菌株(strain)、化合物(chemical)、培养基(medium)、温度(temperature)、处理时间(time)	模型真正需学习的因果变量	编码进生物状态 latent
测量过程	数据来源(data_source)、仪器(instrument)、实验板(plate)	解释系统偏差，不代表生物机制	单独进入仪器校准分支
标识与划分	sample_ID、product_id（含 quality control）、split_final	样本对齐与划分	不参与建模
待确认	其余约 4 个字段	可能为 replicate ID、批次号等	数据发布后确认

稳定性关键：将测量过程字段混入生物条件编码，会导致模型学到仪器偏好而非生物规律，评测时泛化崩溃。

3.1.2a 两个榜单：封闭数据榜 vs 开放知识榜

虚拟细胞方向内设**两个榜**，分别排名：

榜单	允许使用的数据	适合的参赛者	评分方式
封闭数据榜	仅组委会发布的训练数据、字段字典与官方基线	不想处理外部数据、专注模型本身的团队	公平比较下从内部数据学习规律的能力
开放知识榜	封闭数据 + 公开数据库、文献知识、通路模型与工具调用（须完整记录外部资源）	希望结合先验知识、科学推理或智能体工具链的团队	鼓励结合先验知识，要求结果可复现

建议：初赛以封闭数据榜为主（不依赖外部数据匹配，可行性高）；复赛/决赛可尝试开放知识榜（引入基因组/SMILES 外部表征，详见 5.3~5.4）。两个榜分别评分，但二者如何进入统一总排名尚待确认（参赛指导建议封闭数据榜为主排名输入）。

跨方向统一排名提示：算法赛题**不按方向分设排名**，而是以统一评审标准对虚拟细胞、小分子轨迹、材料智能体三个方向同台评分、统一排名，全赛道最终只决出**一名**算法赛题冠军。各方向内量化成绩先归一化为可比分数（参照小分子方向 0-100 归一化口径推断），再按 45/30/20/5 评审维度统一打分。这意味着你的方案不仅要在虚拟细胞方向内领先，还要在科学意义与创新性上有跨方向的竞争力。

3.1.3 验证场景（实体级正交划分）

为检验真正的实体级泛化，验证集按「菌株是否见过 × 化合物是否见过」正交拆分，而非随机划分：

验证场景	菌株	化合物	样本数	检验能力	难度
val_chem_only	已见	未见	1,065	结构信息能否迁移到新化合物	中
val_strain_only	未见	已见	1,547	遗传背景表征能否迁移到新菌株	中
val_both	未见	未见	269	双重零样本预测（最难）	高
val_time	按时间划分	按既定条件	157	跨处理时间外推	中

验证纪律：内部研究必须**分场景分别报告**，不能只给一个由大样本场景（val_strain_only 1,547 样本）主导的总分。val_both（269 样本）是区分模型泛化能力的试金石。

3.1.4 方案选型对比（为什么选 MLP 作为 Baseline）

核心理念：不预设唯一方案，而是列出所有候选方案，从「可行性、稳定性、泛化能力、实现成本」四个维度对比。借鉴以往类似比赛的经验，**先用最简方案跑通流水线，再根据实际结果决定是否切换更复杂方案。**

3.1.4a 类似比赛经验借鉴

本赛题本质是**高维多输出回归**（输入：离散条件；输出：4,232 维连续向量），与以下四类比赛在任务结构上有相似性。以下经验均为同类比赛中被反复验证的做法，每条附「在本赛题的落地方式」。

类型一：细胞扰动响应预测类（相似度最高）

代表比赛：[Open Problems – Single-Cell Perturbations](#) (Kaggle · NeurIPS 2023) ——小分子扰动 → 18,211 维基因表达响应，要求泛化到**未见细胞类型 × 未见化合物**组合（与本赛题的菌株 × 化合物正交划分几乎同构）。

任务相似点：扰动条件 → 细胞分子响应；输出数千至数万维；按「实体是否见过」划分验证。

可借鉴经验	为什么有效	本赛题落地
实体表征是泛化的关键	one-hot 在新实体场景完全失效（所有新实体都是同一个「未知类别」）；获奖方案普遍用外部知识构建实体表征（基因→序列/功能注释，化合物→结构）	提分阶段用基因组/SMILES 表征（ 5.3.1 、 5.3.2 ）；Baseline 阶段先用统计先验特征过渡（ 3.1.5c ）
均值/对照基线极强	高维生物数据自相关性极强，「不预测」的简单基线就能拿到很高分数；评委更看重 相对基线 的提升而非绝对分	本赛题 Matched Control 逐蛋白 R ² 已达 0.72~0.84（ 4.2.3 ），模型必须证明自己在此之上还有提升
分场景验证是标配	随机切分让同一实体同时出现在训练/验证中，指标虚高，选出的模型在真实 OOD 场景崩盘	内部验证按 (strain, chemical) 分桶（ 5.6.3 ），分场景分别报告（ 3.1.3 ）
多任务共享表征	高维输出若每维独立建模，参数量爆炸且忽略输出间相关性；共享底层编码让所有输出维度互相正则	条件编码 → 共享 MLP → 4,232 维线性输出头（ 4.3.4 ）

类型二：化合物→生物响应预测类（作用机制/活性预测）

代表比赛：[Mechanisms of Action \(MoA\) Prediction](#)（Kaggle · 2020）——772 维基因表达 + 100 维细胞活性 → 206 个作用机制多标签分类；含 **vehicle 溶剂对照样本**（类似本赛题 DMSO 对照），提供 drug_id 供按化合物分组验证。

任务相似点：化合物 → 高维生物响应；输出标签间强相关（同通路分子共变）。

可借鉴经验	为什么有效	本赛题落地
结构表征质量 > 模型复杂度	化合物类比赛的共识：SMILES→指纹/描述符/GNN「三件套」做扎实，简单模型也能赢；表征错了再深的网络也救不回	名称→SMILES 标准化须保留 CID 并人工复核（错配=表征完全失效， 5.3.2 ）
标签相关性可利用	输出维度不独立（同通路蛋白共表达），预测间的相关性结构本身就是可学习的信号	蛋白相关性 loss、按通路分组多头（ 5.6.3 、方案 C）
分组多头 > 纯共享	按功能组（通路）分预测头，兼顾共享表征与组内特异建模	方案 C（ 3.1.4c ）；需蛋白功能注释，列为 Baseline 升级版

类型三：分子结构→数值输出类（分子性质预测）

代表比赛：[Predicting Molecular Properties](#)（Kaggle · 2019）——分子三维结构 → 原子间耦合常数回归；获奖方案以 GNN/GCN 为主流，「验证策略」被 top-5 团队列为成功首因。

任务相似点：化学结构 → 连续值预测；特征工程驱动。

可借鉴经验	为什么有效	本赛题落地
多特征融合比单一特征稳	指纹、描述符、预训练 embedding 各有所长，拼接后对单一特征的失效有冗余	化合物高阶特征融合（ 5.3.4b ）
特征统计量必须只在训练集拟合	用全数据算均值/方差是这类比赛最常见的泄漏翻车点	缺失率、标准化统计量仅用训练行（ 4.1.1 、 5.6.4 ）

类型四：多输出 + 系统性缺失比赛

代表比赛：[OpenVaccine: COVID-19 mRNA Vaccine Degradation Prediction](#) (Kaggle · 2020) —— mRNA 序列 → 多目标降解率回归；**仅部分输出维度计入评分**（与 mask-aware 思想一致），且测试集序列比训练集更长，考验长度外推。

任务相似点：输出矩阵含大量缺失；缺失非随机（与仪器灵敏度、丰度相关）。

可借鉴经验	为什么有效	本赛题落地
mask-aware 是硬性要求	缺失填 0 等于告诉模型「该蛋白强度为 0」，引入系统性虚假信号，高维下被放大	mask 矩阵 + 缺失不贡献梯度 (5.2.1 、 4.3.5)
后处理校准零成本提分	高维输出的模型普遍存在系统性偏差（整体偏高/偏低），用参照基线做偏移校准可稳定修正	用 control 基线做偏移校准（ 5.6.3 ）；批次校准分支是该校准的模型内化 (5.4.3)

六条通用经验（跨比赛反复验证）：

- 1. **先简后繁**——复杂模型的 bug 与数据流水线的 bug 难以区分，简单模型能先暴露数据问题。落地：先跑蛋白均值/Matched Control（零训练），再上 MLP，每步可回退（3.1.4d、5.9）。
- 2. **共享 encoder + 多头输出**——高维多输出的标准范式，参数效率与正则效果兼得。落地：Baseline 即采用（4.3.4）。
- 3. **loss 设计比模型结构更重要**——高维回归中 loss 的加权/屏蔽方式直接决定模型学什么，同模型换 loss 差距可达量级。落地：mask-aware MSE → +FC loss 对齐 65% 权重 → 蛋白相关性 loss（5.6.1）。
- 4. **交叉验证策略决定上限**——验证划分方式决定你选出的模型是否真的能泛化。落地：分桶 k-fold，同组不跨 fold（5.6.3）。
- 5. **后处理校准值得做**——零训练成本，稳定提升 FC 类指标。落地：control 偏移校准（5.6.3）。
- 6. **集成不是必须**——高维输出的集成方差大、调试成本高；单模型做好正则+验证往往比草率集成更稳。落地：优先单模型 + 固定种子；TTA 是更轻的替代（5.6.3）。

3.1.4b 候选方案一览

基于上述经验，本赛题的候选方案（均为神经网络路线）：

#	方案	类型	核心思路	优势	劣势
A	蛋白均值 / Matched Control	统计基线	不建模，直接输出统计量	零训练、最稳定	无法
B	条件编码 MLP	神经网络	one-hot/embedding → MLP → 4,232 维	共享表征、训练快、易扩展	需调无法
C	共享 Encoder + 多头输出	神经网络	底层共享条件编码，上层按蛋白分组多头预测	蛋白间共享特征+分组建模	架构注释
D	Transformer / 注意力交互	神经网络	实体间 self-attention 交互	建模交互强、可加外部表征	参数定、
E	残差分解网络	神经网络（提分）	显式分解共享+特异+交互	直接对齐 65% 的 Δ 指标	需外段用

3.1.4c 各方案详细分析

方案 A：统计基线（必做，非「建模」）

- 用途：验证数据流水线、对齐赛题报告指标
- 实现：见 4.2.1 基线一 和 4.2.2 基线二
- 结论：**所有建模方案都必须超越 Matched Control**，否则模型无意义

方案 B：条件编码 MLP（推荐 Baseline）

- 思路：条件 one-hot/embedding → 2~3 层 MLP → 4,232 维输出
- 核心优势：
 - **共享表征**：所有蛋白共享同一条件编码，参数效率高（<1M 参数）
 - **训练快**：单卡几分钟到十几分钟
 - **易扩展**：后续可平滑升级为残差分解、Transformer 等
 - **mask-aware loss 天然支持**：缺失值不贡献梯度
- 选型理由：在可行性、稳定性、实现成本上均为最优，且为提分阶段的架构演进提供基础
- 借鉴类似比赛经验：扰动预测类比赛中获奖方案多数以 MLP 为起点
- 详细实现：见 4.3 方案 Demo

方案 C：共享 Encoder + 多头输出（可选 Baseline）

- 思路：底层共享条件编码，上层按蛋白功能分组（如代谢/翻译/应激）多头预测
- 核心优势：蛋白间共享特征 + 分组建模，比纯 MLP 更有结构
- 适用条件：需要蛋白功能注释（如 GO term）来分组
- 借鉴类似比赛经验：作用机制预测类比赛中共享 encoder + 多头是标准范式
- 结论：如果数据到手后有蛋白功能注释，可作 Baseline 升级版

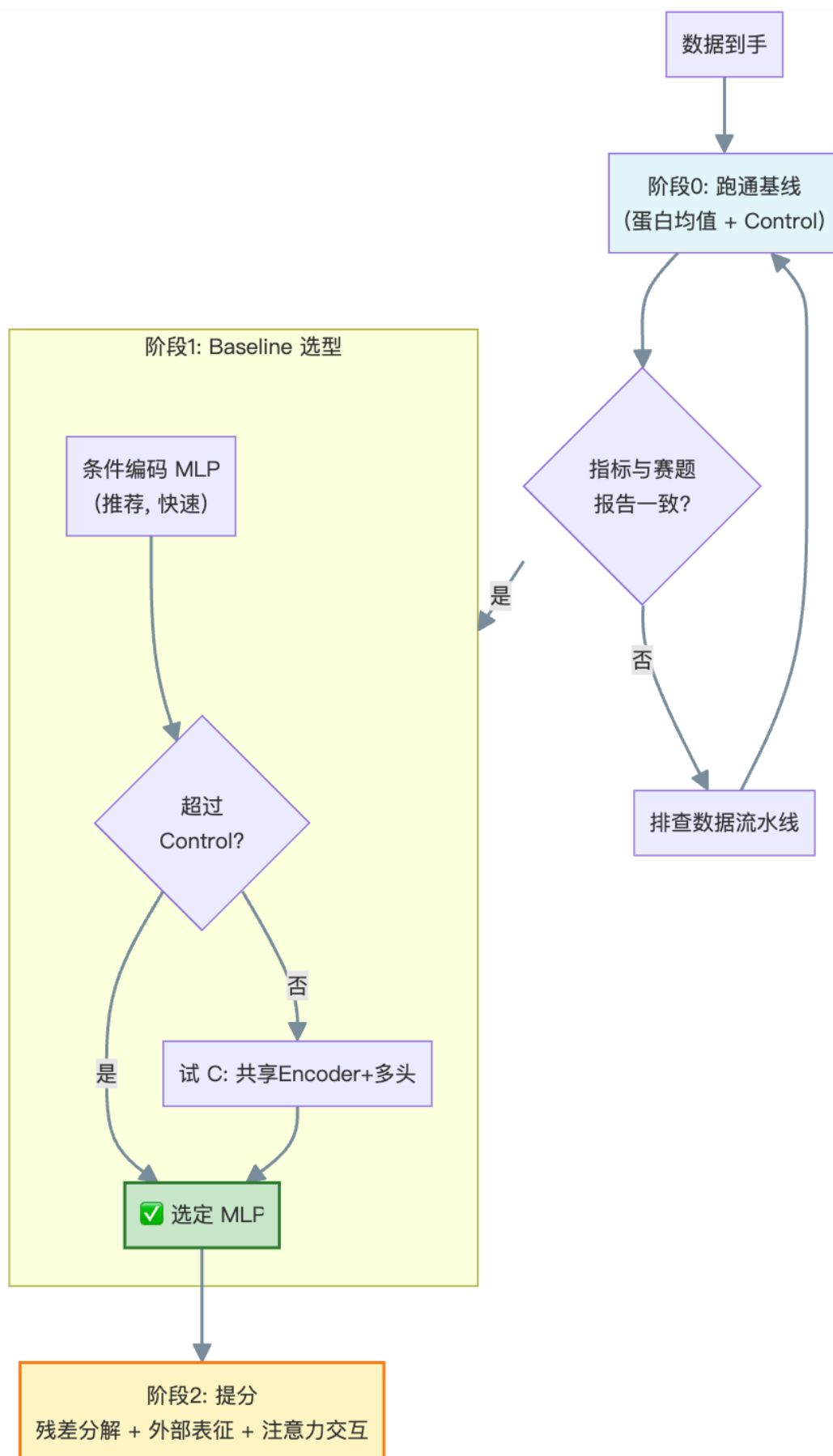
方案 D：Transformer / 注意力交互

- 思路：菌株、化合物、条件分别 embedding 后做 self-attention 交互
- 适用阶段：**提分阶段**（需外部 embedding 才能发挥优势）
- 风险：参数量大、训练不稳定、数据量不足时易过拟合
- 结论：Baseline 阶段不引入，留到提分阶段且确认 MLP 已收敛后再尝试

方案 E：残差分解网络（提分核心）

- 思路：显式分解为共享响应 + 化合物特异 + 菌株调制 + 交互项
- 直接对齐 65% 的 Δ 指标（FC 25% + 两残差 40%）
- 详细设计：见 5.4.1 残差分解架构

3.1.4d 选型决策流程



核心建议：先跑 A（基线）+ B（MLP，15 分钟），确认 MLP 超过 Control 后进入提分阶段。如果 MLP 效果不理想，尝试 C（共享 Encoder + 多头）。数据到手前，教程以 B（MLP）为默认 Baseline 进行讲解。

3.1.5 特征工程（Baseline 阶段，效果提升的关键）

借鉴类似比赛经验：在高维回归比赛中，特征工程对效果提升的贡献往往大于模型结构。好的特征能让简单 MLP 超过没有特征工程的复杂模型。

3.1.5a 为什么要做特征工程

Baseline 的 MLP 如果只做 one-hot 编码，模型只能「记住」训练中见过的菌株/化合物组合，遇到新实体完全无法泛化。特征工程的核心目标是把离散类别变成有信息量的数值向量，让模型即使在 one-hot 失效的新实体场景下，也能利用特征相似性做合理预测。

3.1.5b Baseline 阶段的特征工程清单

特征类别	具体特征	来源	实现难度	对 Baseline 的价值
条件交叉特征	菌株×培养基、化合物×温度等组合 one-hot	元数据	低	让模型直接学到条件交互，减少 MLP 隐层负担
统计特征	每个菌株的训练集蛋白均值向量	训练集	低	相当于给模型一个「先验预测」，MLP 只需学残差
统计特征	每个化合物的训练集 Δ 均值（处理－对照）	训练集	低	直接对齐 FC 指标，让模型先学到「这个药一般会怎么影响蛋白」
统计特征	每个菌株×化合物的训练集均值（若有重复）	训练集	中	最强的先验，但新实体场景无此特征
化合物基础特征	化合物名称的 hash embedding	元数据	低	比 one-hot 更紧凑，且新化合物也有不同 hash 值
时间特征	时间的 sin/cos 编码（周期性）	元数据	低	比纯 one-hot 更好地表达时间连续性
批次特征	仪器/板号的 one-hot	元数据	低	帮助模型分离批次效应

3.1.5c 核心特征详解与代码

特征 1：条件交叉特征（借鉴类似比赛经验）

把两个条件的组合作为一个新类别，让模型不用在隐层里学习交互：

代码块

```
1  # 条件交叉特征：菌株×培养基、化合物×温度
2  metadata['strain_medium'] = metadata['strain'] + '_' + metadata['medium']
3  metadata['chem_temp'] = metadata['chemical'] + '_' + metadata['temperature']
4  # 对新类别做 one-hot 或 hash encoding
```

特征 2：菌株/化合物的统计先验（最重要的 Baseline 特征）

借鉴类似比赛中「用 target encoding 给类别特征注入先验」的技巧，为每个菌株/化合物计算训练集统计量作为特征：

代码块

```
1 # 菌株先验：每个菌株在训练集中的蛋白均值向量 (4,232 维)
2 strain_prior = y_log2[train_mask].groupby(metadata.loc[train_mask,
   'strain']).mean()
3 # 对每个样本，查表得到其菌株的先验蛋白向量
4 X_strain_prior = strain_prior.loc[metadata['strain']].values # (N, 4232)
5
6 # 化合物  $\Delta$  先验：每个化合物的平均扰动效应（处理-对照）
7 # 对训练集处理样本，按化合物分组算  $\Delta$  均值
8 treatment_train = metadata.loc[train_mask &
   (metadata['product_id']=='treatment')]
9 delta_train = y_log2.loc[treatment_train.index] -
   y_log2.loc[matched_control.index].values
10 chem_delta_prior = delta_train.groupby(treatment_train['chemical']).mean()
11
12 # 将先验向量拼接条件编码后面
13 X_augmented = np.concat([condition_onehot, X_strain_prior,
   X_chem_delta_prior], axis=1)
```

关键：统计先验特征在**已见实体**上效果极好，但在**新实体**上无此特征（新菌株/新化合物没有训练集统计量）。此时可用全局均值填充，或留到提分阶段用外部表征替代。

特征 3：时间周期性编码

时间点有内在连续性（2h→6h→24h），用 sin/cos 比纯 one-hot 更好：

代码块

```
1 # 假设时间点为 [2, 6, 12, 24, 48, 72] 小时
2 time_hours = metadata['time'].str.extract('(\d+)').astype(float).values
3 # 归一化到 [0, 2 $\pi$ ]
4 time_norm = 2 * np.pi * time_hours / time_hours.max()
5 time_sin = np.sin(time_norm)
6 time_cos = np.cos(time_norm)
```

特征 4：化合物名称 hash encoding

对新化合物也能给出不同的特征值（虽然无语义信息，但比 one-hot 的「全零」好）：

代码块

```
1 # 化合物名称 hash → 固定维度向量
2 def hash_encode(name, dim=32):
3     h = hashlib.md5(name.encode()).hexdigest()
4     vec = np.zeros(dim)
5     for i in range(dim):
```

```
6         vec[i] = int(h[i*2:i*2+2], 16) / 255.0
7     return vec
8
9     X_chem_hash = np.array([hash_encode(name) for name in metadata['chemical']])
```

3.1.5d Baseline 特征工程的优先级

建议按以下顺序逐步加入特征，每加一组对比一次效果：

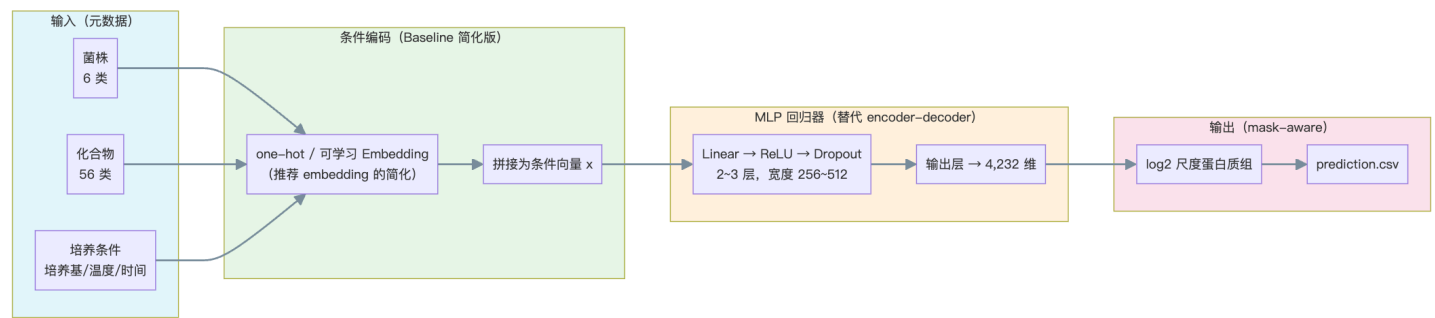
优先级	特征	预期效果	风险
P0（必做）	条件 one-hot	基础	无
P1（强烈推荐）	菌株/化合物统计先验	已见实体大幅提升	新实体无此特征，需 fallback
P2（推荐）	条件交叉特征	交互建模更高效	维度增加
P3（可选）	时间 sin/cos	时间外推更平滑	效果有限
P4（可选）	化合物 hash	新化合物有非零特征	无语义信息

3.1.6 总体技术架构（基于选型结论）

基于 3.1.4 的对比分析，Baseline 阶段推荐方案 B（条件编码 MLP），理由：在可行性、稳定性、实现成本上最优，且为提分阶段的架构演进（残差分解、Transformer）提供基础。

赛题推荐的建模思路为：把菌株、化合物、实验条件分别编码 → 模型学习交互 → 输出蛋白质组预测（考虑仪器偏差）。本方案以该框架为基础：

- Baseline 阶段：将推荐框架简化为最简可验证版本——「条件编码 → MLP 回归 → mask-aware 输出」三步架构，刻意避免引入外部表征等在数据到手前无法验证的模块。
- 提分阶段：逐步实现推荐框架的完整版本（encoder-decoder + 仪器变量），并在此基础上增强（残差分解、外部表征、蛋白先验等，详见第五章）。



与推荐框架的对应关系：

推荐框架组件	Baseline 简化实现	提分阶段完整实现（第五章）
菌株/化合物/条件编码	one-hot / 可学习 Embedding	外部可迁移表征（基因组/ SMILES）
学习交互	MLP 隐层	显式 encoder + 交互模块
输出蛋白质组	MLP 输出层	残差分解输出
考虑仪器偏差	暂不引入（靠 log2 + mask 缓解）	仪器校准分支

极简架构的设计理由（可行性/稳定性优先）：

设计选择	理由	风险评估
one-hot/embedding 起步，不引入外部表征	外部基因组/SMILES 匹配有不确定性，Baseline 阶段先验证流水线	one-hot 无法泛化到新实体，但可作可信下界
端到端 MLP，不做残差分解	残差分解需调超参，Baseline 阶段不增加调参负担	指标上限有限，但训练稳定、易复现
不引入批次校准分支	校准分支需与生物编码解耦设计，复杂度高	可能受批次效应影响，但可通过 log2 + mask 缓解
仅 2~3 层 MLP，宽度 256~512	参数量小（<1M），单卡几分钟可训练	拟合能力有限，足以超过蛋白均值基线即可

核心原则：Baseline 的目标是**把流水线跑通、把基线对齐、把验证做扎实**，而非追求最高分。推荐框架的完整实现在提分阶段逐步引入，每个组件可独立验证。

方案切换提示：如果数据到手后发现 MLP 效果不理想，可参考 3.1.4 尝试方案 C（共享 Encoder + 多头输出）。代码骨架设计为模块化，条件编码和 mask-aware loss 部分可复用。

3.1.7 稳定性设计 —— 三层防护（数据/模型/评估）

蛋白质组数据天然存在高噪声、批次效应和缺失值，Baseline 需内置稳定性机制：

防护层	策略（Baseline 已内置）	作用
数据层	mask-aware 全流程、仅训练集统计量、缺失率仅从训练行计算	防止缺失值泄漏与分布偏移
模型层	端到端 MLP（低方差）、Dropout 正则、固定随机种子	降低训练方差，保证可复现
评估层	分场景验证、多指标联合、与两类基线持续对比	防止单一指标欺骗，确认提升来源

3.2 预期方法路线

按「Baseline 优先跑通 → 提分阶段按需引入复杂模块」的渐进式路线推进。Baseline 阶段做三件事：**跑通流水线 + 多方案对比 + 对齐基线**；提分阶段才引入外部表征、残差分解、交互模块等创新点。

阶段 0 | 数据对齐与基线 (Baseline, 必须)

- 以 `sample_ID` 对齐元数据与蛋白矩阵 (不依赖行顺序)
- 仅用训练行计算缺失率, 删 $\geq 80\%$ 缺失蛋白 (5,243 $\rightarrow$ 预期 4,232)
- 构建 mask 矩阵 + log2 尺度转换
- 跑通蛋白均值基线与 Matched Control 基线, 完成第一次端到端提交
- **验证点:** 基线指标与赛题报告一致 ($\pm 5\%$); 提交格式、尺度声明、样本数、蛋白列数与 feature contract 一致
- **回退点:** 若基线指标与赛题报告不符, 排查对齐/过滤/尺度环节 (无回退, 必须通过)

阶段 1 | 条件编码 MLP (Baseline 主方案)

- 菌株/化合物/上下文 one-hot 或可学习 embedding 编码
- 端到端 MLP 回归 (2~3 层, 宽度 256~512)
- mask-aware MSE loss 训练
- 如果效果不理想, 尝试共享 Encoder + 多头输出 (方案 C)
- **验证点:** 逐蛋白 R^2 中位数是否转正 (>0), 超过 Matched Control 基线
- **回退点:** 若 MLP 不收敛或未超过 Control, 检查特征工程或回退到 Control 基线

阶段 0~1 即为初赛可提交的 **Baseline**。以下阶段为提分内容, 详见第五章。

阶段 2~4 | 提分阶段 (详见第五章)

- 阶段 2: 实体可迁移表征 (菌株基因组 / 化合物 SMILES) $\rightarrow$ 提升 `val_both` / `val_strain_only`
- 阶段 3: 残差分解 + 交互模块 + 批次校准 + 蛋白先验 $\rightarrow$ 提升 fold change、残差类指标
- 阶段 4: 科学意义解读 + 创新范式 (生成式 / 主动学习) $\rightarrow$ 拉开科学意义与创新性分数

3.3 数据来源、依赖工具与运行流程

3.3.1 数据来源

数据	来源	用途	可获取性
比赛数据	WAYB/WAYC 比赛数据文件（官网发布）	训练/验证/测试元数据与蛋白组	报名后开放
菌株基因组	Peter et al. (2018), <i>Nature</i> 556:339–344, 1,011 株酵母群体基因组	菌株 embedding	公开可下载
S288C 参考基因组	SGD (Saccharomyces Genome Database)	DHY210 代理基因组	公开可下载
化合物结构	名称→SMILES 标准化 (PubChem / ChEMBL)	化合物 embedding	公开可查询
蛋白先验	GO / KEGG / STRING 等公开数据库	蛋白侧先验约束	公开可下载

3.3.2 依赖工具

层级	工具	用途
语言/框架	Python 3.10+、PyTorch 2.x / JAX、numpy、pandas、scipy	核心计算
生信/化学	RDKit (SMILES/Morgan FP) 、PyG/DGL (分子图)、biopython	分子/基因组特征
基因组表征	k-mer 计数 (jellyfish/自写)、genome language model (DNABERT-2 等)	菌株 embedding
实验追踪	MLflow / DVC / wandb、固定随机种子	可复现保障
环境固化	Docker、conda-lock、requirements.txt	复赛提交
评估可视化	matplotlib、seaborn、plotly	结果分析

四、阶段性实验结果或可行性验证

4.1 可行性分析

4.1.1 Baseline 可行性论证

（1）技术可行性 —— 仅用成熟组件

Baseline 模块	技术成熟度	风险	保障措施
数据预处理 (sample_ID 对齐 / 缺失过滤 / log2 / mask)	成熟 (numpy/pandas 标准操作)	极低	赛题报告已给出完整流程, 可直接复现
条件编码 (one-hot / Embedding)	成熟 (PyTorch 标准层)	极低	业界标准架构, 无需调参
MLP 回归器 (2~3 层)	成熟	极低	参数量 <1M, 训练稳定
mask-aware MSE loss	成熟	极低	公式简单, 无超参
分桶验证脚本	成熟	极低	仅按 split_final 字段分组计算指标

Baseline 不包含任何「中风险」或「高风险」模块（如残差分解、外部表征、交互模块、批次校准）。这些模块统一放在第五章提分阶段，在基线稳定后逐项验证引入。

(2) 数据可行性 —— 任务规模可控

- 8,958 训练+验证样本，5,920 训练样本，4,232 维输出
- 对端到端 MLP 而言属于**高维小样本**场景，但 Baseline 目标是超过蛋白均值基线（逐蛋白 R^2 中位数从负转正），而非逼近性能上限
- 蛋白质组的缺失值结构已被赛题报告充分描述，mask-aware 策略可处理
- 正交验证划分已冻结，评测边界清晰，不会出现随机划分式的虚假高分

(3) 资源可行性 —— 单机即可完成

资源	Baseline 需求	提分阶段需求（参考）
计算资源	单机 CPU 即可预处理；单卡 GPU（任意型号）几分钟可训练	单卡 A100/V100，训练 1~2 小时
存储	<1 GB（原始数据 + checkpoint）	<10 GB（含外部基因组/ SMILES）
外部数据	无需任何外部数据	1,011 基因组 / PubChem SMILES
付费 API	无	无

4.1.2 Baseline 稳定性保障

维度	保障措施	验证方法
训练可复现	固定随机种子（Python / numpy / PyTorch）	同种子下两次训练 loss 曲线一致
数据一致性	mask 逻辑在训练/验证/测试统一；统计量仅来自训练集	检查 mask 矩阵与原始 NA 分布一致
评估一致性	分桶验证脚本与赛题报告口径一致	基线指标与赛题报告一致（±5%）
提交一致性	样本数 / sample_ID / 蛋白列 / 尺度声明 四项校验	提交前 assert 通过

4.1.3 Baseline 准确性保障

维度	保障措施	防范的风险
防信息泄漏	缺失率仅从训练行计算；标准化统计量仅来自训练集；不读取测试真值	验证指标虚高、泛化评估失真
防指标欺骗	分桶报告（新化合物/新菌株/双重未知/时间外推）+ 多指标联合（Global R ² / 逐蛋白 R ² / fold change）	被 Global R ² 主导、掩盖泛化短板
防归因模糊	每步与蛋白均值、control 两类基线对比	无法判断提升来自模型还是数据相似性

各阶段验证标准与回退方案见 3.2 预期方法路线 与 5.9.1 每阶段可行性自检表。

4.2 基线可行性验证

初赛虽不强制提交代码，但**附初步实验或可行性验证是加分项**，也能在复赛时节省时间。本节给出两类基线的可行性验证与诊断结果（数据到手即可复现）。

4.2.1 基线一：蛋白均值基线

对每个保留蛋白计算训练集非缺失 log2 均值，对所有样本输出同一向量。不读取测试真值、不需任何外部实体特征，适合做第一次端到端提交。

代码块

```
1  # ===== 基线一：蛋白均值基线（数据到手后套用） =====
2  import numpy as np
3  import pandas as pd
4
5  # 1. 对齐：以 sample_ID 为唯一键（不依赖行顺序）
6  metadata = metadata.set_index('sample_ID')
7  proteome = proteome.set_index('sample_ID')
8
```

```

9  # 2. 缺失过滤：仅用训练行计算缺失率
10 train_mask = metadata['split_final'].eq('train')
11 missing_rate = proteome.loc[train_mask].isna().mean(axis=0)
12 keep_proteins = missing_rate < 0.80          # 保留缺失率 < 80% 的蛋白 (5,243 → 4,232)
13
14 # 3. log2 转换 (缺失位置保持 NaN)
15 y_log2 = np.log2(proteome.loc[:, keep_proteins])
16
17 # 4. 计算训练集每个蛋白的 log2 均值
18 protein_mean = y_log2.loc[train_mask].mean(axis=0)
19
20 # 5. 对所有测试样本输出同一向量
21 test_mask = metadata['split_final'].eq('test')
22 prediction = np.tile(protein_mean.values, (test_mask.sum(), 1))
23
24 # 6. 保存提交文件 (声明 prediction_scale=log2)
25 pd.DataFrame(prediction,
    columns=keep_proteins[keep_proteins].index).to_csv('prediction.csv')

```

4.2.2 基线二：Matched Control 基线

对每个处理样本查找同一数据来源、仪器、实验板、菌株、培养基、温度、处理时间下的 Water/DMSO 对照，以这些 matched control 的 log2 蛋白质组均值作为预测。该基线利用赛题元数据中的对照组信息，代表「不建模」情况下的最强 baseline。对比此基线，验证模型是否有意义，模型是否真的学到了化合物的扰动效果，如果模型不如 Control，说明模型没学到任何东西。

4.2.3 当前数据上的诊断结果

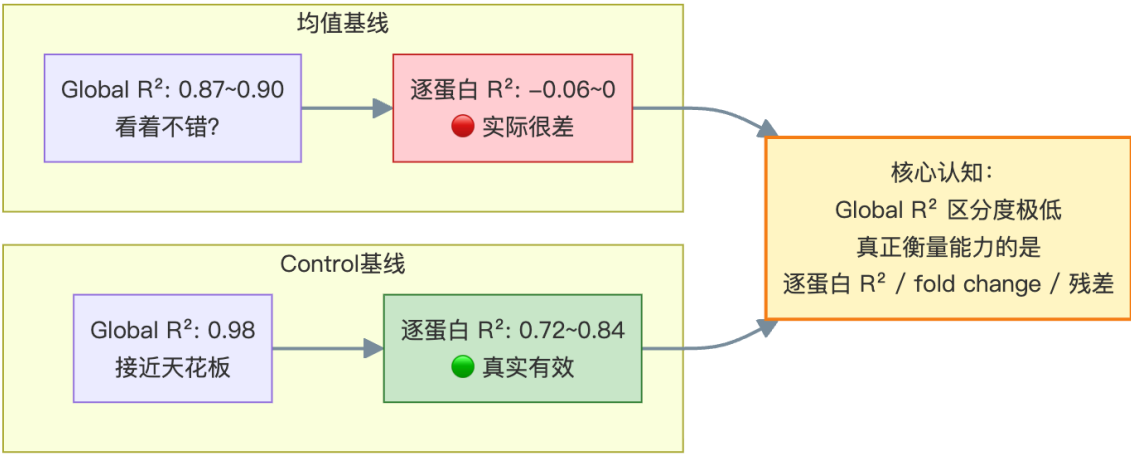
下表为冻结验证划分 + 训练集缺失过滤 + log2 mask-aware 指标的诊断结果（非正式榜单成绩，来自于赛题解读报告）：

场景	基线	样本数	log2 RMSE	Global R ²	蛋白 R ² 中位数
双重未知	蛋白均值	266	0.994	0.871	-0.064
双重未知	Matched control	266	0.382	0.98	0.809
新化合物	蛋白均值	1,015	1.004	0.868	-0.038
新化合物	Matched control	1,015	0.379	0.98	0.836
新菌株	蛋白均值	1,293	0.875	0.897	-0.036
新菌株	Matched control	1,293	0.399	0.978	0.726
时间验证	蛋白均值	128	0.869	0.9	-0.009
时间验证	Matched control	128	0.426	0.975	0.719

样本数差异说明：上表样本数（如新菌株 1,293）小于验证集样本数（如 val_strain_only 1,547），原因是基线诊断仅在**能找到 exact matched control 且处理真值与对照均非缺失**的样本子集上计算。这并非数据缺失，而是 Matched Control 基线的计算约束。

4.2.4 关键诊断结论

一图看懂基线差异（为什么不能只看 Global R²）：



1. **数据相似性极高，Global R² 区分度极低：**蛋白均值基线的 Global R² 已达 0.87~0.90，control 基线达 0.98。这意味着**不能只用 Global R² / profile 相关性评判模型**——所有模型在这个指标上都会接近天花板。
2. **真正的区分度在「差异维度」指标：**逐蛋白 R² 中位数在均值基线下为**负值**（-0.064~-0.009），control 基线将其提升到 0.72~0.84。后续模型需证明自己能在 control 基础上继续提升。
3. **control 基线是强基线，但并非不可超越：**control 已经解释了大部分实验背景和基础生物状态，剩余空间是**化合物特异响应**——这正是 fold change PCC、残差类指标的设计目标。

4. 基线差异性验证了指标设计的合理性：蛋白均值 vs control 的逐蛋白 R^2 差距巨大（-0.064 vs 0.809），说明指标能有效区分模型能力。
5. 后续模型的正确目标：不是比 Global R^2 （大家都接近 1），而是比逐蛋白 R^2 、fold change、残差类指标——这些指标才真正衡量「模型学到了什么生物学」。

4.3 方案 Demo：从模拟数据到端到端提交

以下 Demo 使用**模拟数据**（非真实赛题数据），演示完整的数据处理 → 模型训练 → 预测提交流程。真实数据到手后，只需替换数据路径即可运行。初赛不强制提交代码，但理解此 Demo 有助于撰写方案文档中的技术路线部分。

4.3.1 生成模拟数据

代码块

```
1  # ===== 生成模拟数据（仅用于 Demo，真实数据替换即可） =====
2  import numpy as np
3  import pandas as pd
4
5  np.random.seed(42)
6
7  # 模拟参数（缩小规模以便快速运行）
8  N_STRAINS = 6          # 菌株数
9  N_CHEMS = 10           # 化合物数（真实为 55）
10 N_MEDIUM = 2           # 培养基数
11 N_TEMP = 2             # 温度数
12 N_TIME = 3            # 时间点数
13 N_PROTEINS = 50        # 蛋白数（真实为 5,243，过滤后 4,232）
14 N_SAMPLES = 200       # 样本数（真实为 8,958）
15
16 # 生成元数据
17 strains = [f'STR{i}' for i in range(N_STRAINS)]
18 chemicals = [f'drug_{chr(65+i)}' for i in range(N_CHEMS)] + ['DMSO', 'Water']
19 # 含对照
20 media = ['YPD', 'SD']
21 temps = ['30C', '37C']
22 times = ['2h', '6h', '24h']
23
24 metadata = pd.DataFrame({
25     'sample_ID': [f'S{i:05d}' for i in range(N_SAMPLES)],
26     'strain': np.random.choice(strains, N_SAMPLES),
27     'chemical': np.random.choice(chemicals, N_SAMPLES),
28     'medium': np.random.choice(media, N_SAMPLES),
29     'temperature': np.random.choice(temps, N_SAMPLES),
30     'time': np.random.choice(times, N_SAMPLES),
```

```

30     'data_source': np.random.choice(['WAYB', 'WAYC'], N_SAMPLES),
31     'instrument': np.random.choice(['timsTOF', 'QExactive'], N_SAMPLES),
32     'plate': [f'P{i:02d}' for i in np.random.randint(1, 5, N_SAMPLES)],
33     'product_id': np.random.choice(['treatment', 'solvent_control',
    'quality_control'], N_SAMPLES, p=[0.7, 0.2, 0.1]),
34 })
35
36 # 划分 split_final (简化版)
37 metadata['split_final'] = 'train'
38 metadata.loc[metadata['sample_ID'].str.startswith('S0015'), 'split_final'] =
    'val_chem_only'
39 metadata.loc[metadata['sample_ID'].str.startswith('S0017'), 'split_final'] =
    'val_strain_only'
40 metadata.loc[metadata['sample_ID'].str.startswith('S0019'), 'split_final'] =
    'val_both'
41
42 # 生成蛋白质组数据 (raw intensity, 含 NA)
43 proteome_data = np.random.lognormal(mean=11, sigma=1.5, size=(N_SAMPLES,
    N_PROTEINS))
44 # 随机引入 15% 缺失值
45 mask = np.random.random((N_SAMPLES, N_PROTEINS)) < 0.15
46 proteome_data[mask] = np.nan
47 proteome = pd.DataFrame(proteome_data,
48                         index=metadata['sample_ID'],
49                         columns=[f'protein_{i+1}' for i in range(N_PROTEINS)])
50
51 print(f"元数据: {metadata.shape}, 蛋白质组: {proteome.shape}")
52 print(f"缺失率: {proteome.isna().sum().sum() / proteome.size:.1%}")

```

4.3.2 数据预处理 (对齐 → 过滤 → log2 → mask)

代码块

```

1  # ===== 数据预处理 =====
2  # 1. 以 sample_ID 对齐 (不依赖行顺序)
3  proteome = proteome.set_index(metadata['sample_ID'])
4  metadata = metadata.set_index('sample_ID')
5
6  # 2. 仅用训练行计算缺失率
7  train_mask = metadata['split_final'].eq('train')
8  missing_rate = proteome.loc[train_mask].isna().mean(axis=0)
9  keep_proteins = missing_rate < 0.80    # 保留缺失率 < 80% 的蛋白
10 print(f"过滤: {proteome.shape[1]} → {keep_proteins.sum()} 蛋白")
11
12 proteome_filtered = proteome.loc[:, keep_proteins]
13

```

```

14 # 3. 构建 mask 矩阵 (1=有值, 0=缺失)
15 mask_matrix = ~proteome_filtered.isna()
16
17 # 4. log2 转换 (缺失位置保持 NaN)
18 y_log2 = np.log2(proteome_filtered)
19
20 print(f"预处理后: {y_log2.shape}, 有效值比例: {mask_matrix.values.mean():.1%}")

```

4.3.3 基线一：蛋白均值基线

代码块

```

1 # ===== 基线一: 蛋白均值 =====
2 # 对每个保留蛋白计算训练集非缺失 log2 均值
3 protein_mean = y_log2.loc[train_mask].mean(axis=0)
4
5 # 对所有测试样本输出同一向量
6 test_mask = ~train_mask # 简化: 非训练即测试
7 prediction_mean = np.tile(protein_mean.values, (test_mask.sum(), 1))
8
9 # 计算简单指标 (逐样本 Global R2)
10 y_true_test = y_log2.loc[test_mask].fillna(0).values
11 ss_res = ((y_true_test - prediction_mean) ** 2).sum()
12 ss_tot = ((y_true_test - y_true_test.mean(axis=0)) ** 2).sum()
13 global_r2 = 1 - ss_res / ss_tot
14 print(f"蛋白均值基线 Global R2: {global_r2:.3f} (真实赛题约 0.87) ")

```

4.3.4 基线二：条件编码 MLP (Baseline 模型)

代码块

```

1 # ===== 基线二: 条件编码 MLP =====
2 import torch
3 import torch.nn as nn
4
5 class ConditionMLP(nn.Module):
6     """最简条件编码 MLP: one-hot → 隐层 → 4,232 维输出"""
7     def __init__(self, n_strains, n_chemicals, n_media, n_temps, n_times,
8 n_proteins, hidden=256):
9         super().__init__()
10         # 条件 one-hot 维度
11         dim_in = n_strains + n_chemicals + n_media + n_temps + n_times
12         self.mlp = nn.Sequential(
13             nn.Linear(dim_in, hidden),
14             nn.ReLU(),

```

```

14         nn.Dropout(0.1),
15         nn.Linear(hidden, hidden),
16         nn.ReLU(),
17         nn.Dropout(0.1),
18         nn.Linear(hidden, n_proteins) # 输出 log2 尺度
19     )
20
21     def forward(self, x):
22         return self.mlp(x)
23
24     def encode_conditions(meta_row, strain_list, chem_list, media_list, temp_list,
25                          time_list):
26         """将一行元数据编码为 one-hot 向量"""
27         vec = []
28         vec += [1 if s == meta_row['strain'] else 0 for s in strain_list]
29         vec += [1 if c == meta_row['chemical'] else 0 for c in chem_list]
30         vec += [1 if m == meta_row['medium'] else 0 for m in media_list]
31         vec += [1 if t == meta_row['temperature'] else 0 for t in temp_list]
32         vec += [1 if t == meta_row['time'] else 0 for t in time_list]
33         return torch.tensor(vec, dtype=torch.float32)
34
35     # 初始化模型
36     model = ConditionMLP(
37         n_strains=len(strains), n_chemicals=len(chemicals),
38         n_media=len(media), n_temps=len(temps), n_times=len(times),
39         n_proteins=keep_proteins.sum(), hidden=256
40     )
41     optimizer = torch.optim.Adam(model.parameters(), lr=1e-3)
42     print(f"模型参数量: {sum(p.numel() for p in model.parameters()):,}")

```

4.3.5 mask-aware 训练循环

代码块

```

1  # ===== mask-aware 训练 =====
2  # 准备训练数据
3  train_idx = metadata[train_mask].index
4  X_train = torch.stack([encode_conditions(metadata.loc[sid], strains,
5                                         chemicals, media, temps, times)
6                        for sid in train_idx])
7  y_train = torch.tensor(y_log2.loc[train_idx].fillna(0).values,
8                        dtype=torch.float32) # NA 填 0
9  mask_train = torch.tensor(mask_matrix.loc[train_idx].values,
10                           dtype=torch.float32) # mask 矩阵
11
12  # 训练循环

```

```

10 EPOCHS = 50
11 for epoch in range(EPOCHS):
12     model.train()
13     optimizer.zero_grad()
14
15     pred = model(X_train) # (N, P)
16
17     # mask-aware MSE: 只在有值的位置计算 loss
18     diff = (pred - y_train) ** 2
19     loss = (diff * mask_train).sum() / mask_train.sum()
20
21     loss.backward()
22     optimizer.step()
23
24     if (epoch + 1) % 10 == 0:
25         print(f"Epoch {epoch+1}/{EPOCHS}, mask-aware MSE: {loss.item():.4f}")
26
27     # 预测测试集
28     model.eval()
29     test_idx = metadata[test_mask].index
30     X_test = torch.stack([encode_conditions(metadata.loc[sid], strains, chemicals,
31                                         media, temps, times)
32                             for sid in test_idx])
33     with torch.no_grad():
34         prediction_mlp = model(X_test).numpy()
35
36     print(f"MLP 预测完成: {prediction_mlp.shape}")
37     print(f"预测值范围: [{prediction_mlp.min():.2f}, {prediction_mlp.max():.2f}]
38           (log2 尺度) ")

```

4.3.6 生成提交文件

代码块

```

1  # ===== 生成 prediction.csv =====
2  submission = pd.DataFrame(
3      prediction_mlp,
4      index=test_idx,
5      columns=y_log2.columns
6  )
7  submission.index.name = 'sample_ID'
8
9  # 保存
10 submission.to_csv('prediction.csv')
11 print(f"提交文件已保存: prediction.csv")
12 print(f"  样本数: {len(submission)} (真实赛题为 4,454) ")

```

```
13 print(f" 蛋白列数: {submission.shape[1]} (真实赛题为 4,232) ")
14 print(f" 尺度: log2 (在提交信息中声明 prediction_scale=log2) ")
15
16 # 校验
17 assert not submission.isna().any().any(), "存在 NA! "
18 assert np.isfinite(submission.values).all(), "存在 inf! "
19 print("校验通过: 无 NA、无 inf")
```

4.3.7 Demo 与真实赛题的对应关系

Demo 参数	真实赛题	说明
6 菌株	6 菌株 (4 训练 + 1 验证 + 1 测试)	一致
10 化合物	55 化合物 (38 训练 + 6 验证 + 11 测试)	Demo 缩小
50 蛋白	5,243 → 过滤后 4,232	Demo 缩小
200 样本	8,958 训练验证 + 4,454 测试	Demo 缩小
15% 缺失	约 20% (过滤前)	接近
50 Epochs	视收敛情况, 通常 100~200	真实数据更大
hidden=256	256~512	一致
预测 log2	提交 log2 (建议)	一致

核心要点：Demo 中的代码骨架在真实数据上**无需修改逻辑**，只需替换数据路径和调整超参（如 hidden 维度、epochs）。mask-aware loss、sample_ID 对齐、缺失率过滤、log2 转换等核心逻辑完全一致。

五、提分教程

本章面向复赛/决赛冲刺。算法赛评审权重：**技术性能 45% + 科学意义 30% + 方法创新性 20% + 开源贡献 5%**。提分策略围绕四个维度展开，并从**可行性、稳定性、创新性、准确性**四个角度给出差异化策略。

5.1 提分总览：评分维度拆解与提分地图

5.1.1 评分权重与提分杠杆

评分维度	权重	核心提分点	难度	可行性	对总分的边际贡献
技术性能	45%	数据预处理 + 实体表征 + 模型架构 + 训练策略	中	高	基本盘，每提升 5% 指标 $\approx$ 2.25 分
科学意义	30%	生物学解读 + 机理验证 + 外部数据虚拟实验	高	中	拉开差距，清晰阐述即有 15-20 分
方法创新性	20%	新表征/新架构/新范式	高	中	一个创新点可拿 10-15 分
开源贡献	5%	代码/模型/管线可复用	低	高	保底分，做齐即可拿满

5.1.2 评测指标体系（技术性能内部，精确权重）

指标设计逻辑：传统 Pearson correlation 在单细胞数据上能到 0.999，数据自相关性太高，**无法区分模型好坏**。因此主办方设计了 6 个评分模块，每个模块针对一种具体能力，权重加总 100%。

关键认知：参赛者只需预测绝对丰度 $\hat{y}_{treat}$ 。所有 Δ (fold change) 类指标由评测服务器自动计算： $\Delta_{pred} = \hat{y}_{treat} - y_{control}$ ，其中 $y_{control}$ 是真实对照样本的蛋白组值（由主办方持有，不开放）。参赛者不需要自己预测或提交 fold change。

6 个评分模块（权重已精确化）：

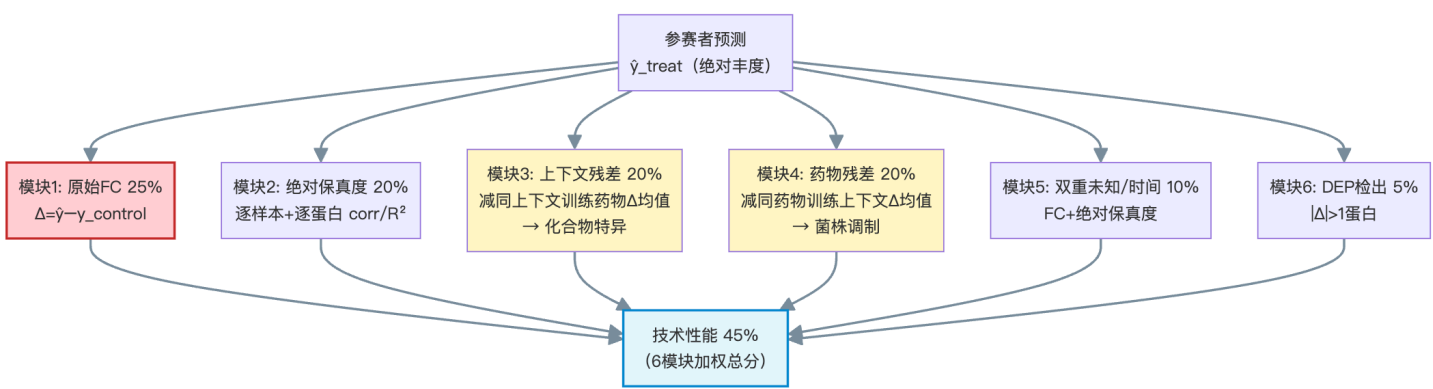
#	评分模块	权重	计算方式（精确版）	适用划分	提分
1	匹配对照原始 FC	25%	$\Delta_{pred} = \hat{y}_{treat} - y_{control}$; $\Delta_{true} = y_{treat} - y_{control}$; 算 $PCC(\Delta_{pred}, \Delta_{true})$ 。对照须匹配来源/菌株/培养基/温度/时间/仪器/板号，匹配到 DMSO 或 Water	所有 OOD 划分	权重化合实蛋
2	绝对保真度	20%	逐样本 $corr/R^2$ （每个样本的预测向量对真值向量）+ 逐蛋白 $corr/R^2$ （每个蛋白跨样本）。PCC 刻画谱形状， R^2 刻画幅度。样本轴与蛋白轴分别聚合	全部划分	预测能只
3	上下文均值残差	20%	μ_{ctx} = 同上下文下训练药物的 Δ_{true} 均值；算 $PCC(\Delta_{pred} - \mu_{ctx}, \Delta_{true} - \mu_{ctx})$ 。「上下文」= 同一菌株/培养基/温度/时间/冻结批次	test_chem_only 等新化合物划分	剔除量，作用
4	药物均值残差	20%	μ_{drug} = 同药物下训练上下文的 Δ_{true} 均值；算 $PCC(\Delta_{pred} - \mu_{drug}, \Delta_{true} - \mu_{drug})$	test_strain_only 等新菌株划分	剔除平均株背
5	双重未知/时间外推	10%	test_both 以原始 FC + 绝对保真度为主；test_time 以绝对保真度 + 原始 FC 为主。 不强制残差指标 （组合外推缺乏稳定训练实体均值）	test_both + test_time	完整向是
6	高效应蛋白与 DEP 检出	5%	对 $ \Delta_{true} > 1$ (log2 空间，即 2 倍变化) 的蛋白，算方向准确率、高效应 PCC、Recall@K、F1/AUPRC。Recall 不单用，须与 precision/F1/AUPRC 组合	全部划分（补充）	权重

晋级门槛与额外评估：

项目	性质	说明
复现与合规	晋级门槛（一票否决）	审查代码、随机种子、归一化流程、外部数据使用与测试泄漏风险
隐藏机理评估	赛后分析/特别展示（非主榜）	从预测蛋白质组推断药物作用机制，或对照公开遗传扰动与药物响应数据评估机理一致性；指标低但机理对可酌情给 特别奖

核心认知：FC（25%）+ 两个残差（40%）= **65% 的分数**围绕「 $\Delta = \text{处理} - \text{对照}$ 」展开，而非绝对丰度。绝对保真度只占 20%。两个残差指标分别考察**化合物特异性**（模块 3）和**菌株背景调制**（模块 4），这直接决定了模型架构设计——需要能分离共享效应与特异效应（见 5.4.1 残差分解架构）。

指标关系图：



Global R² 区分度极低：蛋白均值基线 Global R² 达 0.87，Matched Control 达 0.98——所有模型都接近天花板。真正拉开差距的是模块 1/3/4（共 65%），它们考察的是**化合物特异响应**和**菌株背景调制**。

5.2 数据预处理提分（地基，错了全盘皆输）

数据特性：蛋白质组与转录组不同，missing value 占比比较高。NA 来源主要是仪器灵敏度、蛋白未通过质控、实验条件差异。**不能当 0 处理，也不能独立收集后统一填 0**——会把整个数据分布变得不可预测。稳妥流程：sample_ID 对齐 → 蛋白过滤（仅训练数据计算，>80% 缺失则过滤）→ 保留 mask 矩阵（训练时可填 0 但不贡献梯度）→ log2 尺度。

5.2.1 NA 处理（最易踩坑，影响全局）

蛋白质组 NA 的来源：仪器检测限（信号低于阈值）、肽段覆盖不足（部分肽段未飞上）、实验条件差异（不同批次灵敏度不同）。**NA 不是 0，不应在数据加载后统一填 0**——否则会破坏数据分布，使模型学到虚假的「低表达」信号。

正确流程（四步法）：

- sample_ID 对齐：**以 sample_ID 为唯一键对齐元数据与蛋白矩阵，**不依赖两张表当前的行顺序**

2. 仅用训练行计算缺失率： `missing_rate = proteome[train_idx].isna().mean(axis=0)`，删除 `missing_rate ≥ 0.80` 的蛋白（5,243 → 预期 4,232）
3. 构建 mask 矩阵： `mask = ~proteome.isna()`，保留所有样本的观测 mask。训练时可以填 0，但贡献梯度时不应产生梯度，只在有值的地方计算梯度
4. log2 转换：非缺失 raw intensity 做 log2，缺失位置保持 NaN（训练时填 0 但用 mask 屏蔽梯度）

代码块

```
1 # ===== mask-aware loss 骨架 =====
2 # 核心思想：缺失值填 0 仅用于计算，不贡献梯度
3 mask = ~y_true.isna() # 1=有值，0=缺失
4 y_true_filled = y_true.fillna(0.0) # 缺失位置填 0（不贡献梯度）
5 loss = ((y_pred - y_true_filled) ** 2 * mask).sum() / mask.sum()
```

稳定性说明：mask 机制确保：

- 缺失值不会传播错误梯度
- 训练和验证使用一致的 mask 逻辑
- 样本间有效观测数不同时，loss 自动按有效观测数归一化

5.2.2 尺度纪律（评测红线，不可逾越）

评测要求：提交尺度统一为 **log2 intensity**——raw intensity 不允许直接提交，需参赛者自行转换到 log2 后再提交，并在提交信息中声明 `prediction_scale=log2`（依据：赛题解读直播与分享材料明确「只接受 log2 尺度」）。z-score 等标准化只适合模型内部训练，不能直接作为 prediction.csv 提交；如果模型在标准化空间训练，输出前必须还原到 log2 的生物尺度。

操作	允许范围	禁止
提交尺度	log2 intensity（raw 不允许，需自行转换到 log2 后提交）	raw 直接提交；z-score / 标准化空间直接提交
模型内部	任何标准化（z-score、batch norm 等）	借用验证/测试统计量做标准化
输出前	若在标准化空间训练，必须还原到 log2	提交标准化后的值
尺度声明	必须声明 <code>prediction_scale=log2</code>	不声明 / 声明为 raw

准确性提示：log2 转换后，蛋白强度近似正态分布，适合 MSE 类 loss。raw intensity 通常呈长尾分布，直接建模会放大高丰度蛋白的权重，导致模型偏向拟合高丰度蛋白而忽略低丰度蛋白的生物学变化。

5.2.3 质控样本的巧用

使用提示：metadata 中 `product_id` 含 `quality control` 的样本，化学扰动预测不需要用到，但赛题放进了数据——可以用来衡量批次效应，或为化学扰动下的 fold change 预测做批次校正。

建议用法：

- 计算质控样本在仪器/板间的差异，用于批次校正
- 作为 observation calibration 分支的训练信号
- 评估不同批次间的系统偏差大小

5.2.4 提交文件规范（避免无效提交）

检查项	具体要求	校验方法
样本数	4,454 行，与测试元数据完全一致	<code>len(prediction) == len(test_metadata)</code>
sample_ID	完整、唯一、与测试元数据顺序一致	<code>all(prediction['sample_ID'] == test_metadata['sample_ID'])</code>
蛋白列	名称、数量、顺序与 feature contract 完全一致	与保留蛋白列表逐列比对
预测尺度	log2，声明 <code>prediction_scale=log2</code>	文档中注明
数值合法性	无 NA、无 inf；raw 预测须为正值 > 0	<code>assert not prediction.isna().any().any()</code>
信息边界	生成预测的流程不得读取测试蛋白质组真值	代码审查

5.3 特征工程提分（零样本泛化的核心）

核心问题：one-hot 编码只能记住训练中见过的菌株/化合物；面对新实体，它们都是同一个「未知类别」。模型无法利用「两个新菌株序列是否接近」「两个新化合物结构是否相似」的领域知识。
面向零样本泛化，必须引入实体本身的可迁移表征。

5.3.1 菌株表征（基因组驱动，稳定性关键）

比赛**不直接发布**菌株/化合物 embedding 信息，需自行构建「菌株名 → genome 文件 → 版本 → 来源」映射表。

方法梯度（由易到难，由稳到强）：

方法	复杂度	稳定性	泛化能力	实现方案
k-mer 计数 (k=4~6)	低	高	中	scikit-learn CountVectorizer(analyzer='char', ngram_range=(4,6))
变异位点 / 基因 存在缺失	低	高	中	相对 S288C 参考基因组比对，提取 SNP/indel 矩阵
蛋白序列聚合	中	中	中	将所有基因的蛋白序列拼接后编码
Genome language model embedding	高	中	高	DNABERT-2 / HyenaDNA 等预训练模 型，取 [CLS] token

关键资源与操作步骤：

- 1. **1,011 株酿酒酵母群体基因组**（Peter et al. 2018, *Nature* 556:339–344）：
 - Peter et al. (2018) *Nature* 556:339–344 团队在全世界采集了 1,011 株野生酵母并完成全基因组测序，认为这 1,011 株能代表酿酒酵母这个物种
 - 赛题 6 株酵母即从中**按多样性挑选**最具代表性的 6 株做第一批实验
 - 菌株名为 3 个字母（如 CRD），可在该文对应到全基因组序列
 - 可用 ESM / AlphaFold 类模型对基因组做 embedding，作为菌株特征输入模型
- 2. **DHY210**：
 - 是标准菌株 S288C 的衍生株，**不存在于 1,011 project 文件中**
 - 基因组需用 S288C 作公开代理（SGD 数据库 [yeastgenome.org](https://www.yeastgenome.org)）
 - S288C 是第一株被测序的真核生物全序列，作为 reference
 - **须明确标注这是建模假设，非完全等价**
- 3. 建立映射表时必须保留：**比赛菌株名 → genome 文件名 → 数据库版本 → 下载日期/URL**，确保复现。

创新性提示：如果使用 genome language model embedding 并做详细的消融验证（证明 embedding 信息利用率超越 k-mer），这本身就构成一个有效的方法创新点。

5.3.2 化合物表征（结构驱动，准确性关键）

比赛只提供化合物名称，需自行完成名称标准化与 SMILES 匹配。

最易出错点（不是模型结构，而是数据匹配）：

- 名称歧义：同一化合物可能有多个商品名/俗名
- 盐型差异：盐酸盐 vs 游离碱，SMILES 不同
- 立体异构体：同一分子式但手性不同

- 数据库条目版本差异：PubChem CID 可能因版本而变化

准确性保障流程：

代码块

- 1
- 原始名称 → 名称标准化（去盐、去水合物、统一命名）
- 2
- PubChem / ChEMBL 批量匹配
- 3
- 人工抽检 10% 样本（检查 SMILES 合理性）
- 4
- 生成标准 SMILES
- 5
- 生成表征（Morgan FP / GNN / 预训练表征）
- 6
- 保留完整映射链：原始名称 → CID → SMILES → 来源 → 复核记录

表征方法梯度：

方法	说明	适用场景
Morgan fingerprint（radius=2, 2048 bits）	经典分子指纹，RDKit 一键生成	起点，稳定可靠
分子图神经网络（GNN）	端到端学习分子表示，PyG/DGL 实现	需要较多训练数据
预训练分子表示（MoleBERT/ChemBERTa）	大规模预训练，迁移能力强	小样本场景优势明显

5.3.3 上下文与观测过程特征（稳定性关键）

- 生物上下文：培养基（2 类）、温度（2 类）、处理时间（6 个时间点）→ 编码进生物状态表示
- 观测过程：数据来源、仪器、实验板 → 单独进入仪器校准分支，不与实体表征混用
- 质控样本：metadata 中 product_id 含 quality control，化学扰动预测不需要，但可用于批次校正——计算质控样本在仪器/板间的差异，用于校准

稳定性分析：将观测过程与生物条件分离是本方案的核心稳定性设计。如果混用，模型可能学到「仪器 A 测出的蛋白 X 总是偏低」并当作生物规律，在评测时仪器分布变化导致泛化崩溃。

5.3.4 高阶特征工程（提分阶段，借鉴类似比赛高级技巧）

Baseline 的特征工程（3.1.5）聚焦于统计先验和简单交叉。提分阶段引入更复杂的特征，特别是跨实体、跨组学、结构化先验，进一步提升模型的表达能力。

5.3.4a 蛋白侧特征（模型输出端的先验）

借鉴类似比赛经验：作用机制预测类比赛中，对输出标签（蛋白/机制）之间相关性的建模是核心提分点。本赛题可在蛋白侧引入结构化先验特征。

蛋白特征	来源	作用	实现
蛋白功能通路	SGD/KEGG GO term 注释	同一通路蛋白应协同变化	蛋白→通路映射矩阵，作为多头输出的分组依据
蛋白分子量/疏水性	UniProt	物理属性影响质谱检测偏好	作为蛋白侧特征，帮助仪器校准分支学习
蛋白-蛋白相互作用 (PPI)	BIOGRID/StringDB	相互作用的蛋白应相关	构建邻接矩阵，在 loss 中加入图正则
蛋白丰度先验	S288C 参考蛋白质组	提供蛋白基础表达水平	作为蛋白 bias 的初始化
蛋白功能 embedding	预训练蛋白语言模型 (ESM)	蛋白序列→向量，捕捉功能相似性	作为 decoder 的蛋白 embedding

代码块

```
1  # 蛋白功能 embedding: 用 ESM 提取 4,232 个蛋白的序列特征
2  import esm
3  model_esm, alphabet = esm.pretrained.esm2_t12_35M_UR50D()
4  batch_converter = alphabet.get_batch_converter()
5  # 对每个蛋白序列提取 embedding (35M 模型, 35M 参数)
6  protein_sequences = load_sequences_from_sgd() # 从 SGD 数据库下载
7  protein_embeddings = encode_proteins(model_esm, batch_converter,
8                                     protein_sequences)
9  # protein_embeddings: (4232, 480) 维
9  # 作为 decoder 的蛋白侧 embedding, 与条件 embedding 做交互
```

5.3.4b 化合物高阶特征（超越 SMILES）

Baseline 只用 SMILES → Morgan FP。提分阶段引入更丰富的分子表征。

化合物特征	来源	作用	实现难度
Morgan FP (2048 bit)	RDKit	基础分子指纹	低
MACCS keys (166 bit)	RDKit	药学常用关键子结构	低
分子描述符	RDKit	MW、LogP、TPSA、HBD、HBA 等	低
预训练分子 embedding	ChemBERTa/MolBERT	语法+语义级分子表征	中
图神经网络 embedding	GIN/GAT on molecular graph	捕捉拓扑结构	中高
药物-靶点相互作用	DrugBank/STITCH	已知靶点蛋白→直接关联蛋白组	高（需匹配）

代码块

```
1  # 多种分子特征融合
2  from rdkit import Chem
```



```

3  from rdkit.Chem import AllChem, Descriptors
4
5  def extract_mol_features(smiles):
6      mol = Chem.MolFromSmiles(smiles)
7      # 1. Morgan FP
8      morgan = AllChem.GetMorganFingerprintAsBitVect(mol, 2, nBits=2048)
9      # 2. MACCS keys
10     from rdkit.Chem import MACCSkeys
11     maccs = MACCSkeys.GenMACCSKeys(mol)
12     # 3. 分子描述符
13     descriptors = [
14         Descriptors.MolWt(mol),          # 分子量
15         Descriptors.MolLogP(mol),        # 脂溶性
16         Descriptors.TPSA(mol),           # 拓扑极性表面积
17         Descriptors.NumHDonors(mol),      # 氢键供体数
18         Descriptors.NumHAcceptors(mol),   # 氢键受体数
19     ]
20     return np.concat([morgan, maccs, descriptors])
21
22 # 对 55 个化合物提取特征
23 chem_features = np.array([extract_mol_features(s) for s in smiles_list])

```

5.3.4c 菌株高阶特征（超越基因组）

菌株特征	来源	作用	实现难度
基因组 k-mer (k=3~6)	1,011 project	捕捉序列组成偏好	低
变异位点 VCF	1,011 project	相对于参考株的变异	中
Genome LM embedding	DNABERT-2/AutoVe	预训练基因组表征	中
已知表型注释	SGD/YGAP	菌株已知表型（如药物抗性）	中
系统发育树距离	1,011 project 树文件	菌株间进化距离	低

代码块

```

1  # 菌株系统发育距离作为特征
2  from scipy.spatial.distance import squareform
3  # 1,011 project 提供系统发育树，计算 6 株之间的距离矩阵
4  phylo_dist = compute_phylogenetic_distance(strains, tree_file)
5  # 对每个菌株，距离向量作为特征（需要降维或用 MDS）
6  from sklearn.manifold import MDS
7  mds = MDS(n_components=16, dissimilarity='precomputed')
8  strain_phylo_embedding = mds.fit_transform(phylo_dist)
9  # (6, 16) 维，作为菌株 embedding 的补充

```

5.3.4d 组合特征（实体间交互）

借鉴类似比赛特征工程中的「cross entity feature」——让模型显式看到实体间的关系。

组合特征	含义	实现
菌株×化合物 target encoding	该组合在训练集的平均响应	groupby 后查表
化合物→已知靶点蛋白 mask	该药物直接作用的蛋白（若有）	DrugBank 匹配
菌株→已知敏感药物	该菌株已知的耐药/敏感药物列表	文献挖掘
时间×化合物	同一药物不同时间的响应曲线特征	训练集统计

5.4 模型架构提分（创新性核心区）

本节是创新性的主战场。Baseline 的端到端 MLP 虽然稳定，但无法超越 control 基线太多。真正的提分来自架构层面的创新，每个模块都应对应一个评测指标或一个泛化瓶颈。

5.4.1 创新点一：残差分解架构（对齐评测指标，强烈推荐）

创新动机：评测的「上下文均值残差」（模块 3，20%）和「药物均值残差」（模块 4，20%）本质上就是去除共享效应后的特异响应。加上原始 FC（模块 1，25%），共 65% 的分数围绕 $\Delta = \text{处理} - \text{对照}$ 展开。Baseline 的端到端 MLP 把共享响应和特异响应混在一起预测，无法显式优化这些指标。

创新设计：将预测显式分解为三个可解释组件，直接对齐评测指标：

代码块

```
1   $\hat{y}_{\text{treat}}$  = 共享应激响应（跨化合物共享，对应上下文均值  $\mu_{\text{ctx}}$ ）
2      + 化合物特异残差（由 chemical embedding 驱动，对应模块 3）
3      + 菌株调制残差（由 strain embedding 驱动，对应模块 4）
4      + 菌株 × 化合物 交互项（对应模块 5 双重未知）
```

与评测指标的精确对应：

架构组件	对应评测模块	权重	物理含义
共享应激响应	上下文均值 μ_{ctx} 的基准	—	同一实验条件下所有药物的共同响应（如生长相关蛋白下调）
化合物特异残差	模块 3（上下文均值残差，20%）	20%	减去共享响应后的 化合物特异作用机制
菌株调制残差	模块 4（药物均值残差，20%）	20%	同一药物在不同菌株背景下的 响应调制
交互项	模块 5（双重未知，10%）	10%	菌株 × 化合物的交叉效应
整体绝对丰度	模块 2（绝对保真度，20%）	20%	四组件之和的逐样本/逐蛋白精度
$\Delta = \hat{y} - y_{\text{control}}$	模块 1（原始 FC，25%）	25%	服务器自动计算，只需 $\hat{y}$ 准确

创新性论证：

- **与已有工作的差异：**传统蛋白质组预测采用端到端回归，未显式分离共享与特异成分；本方案首次将残差分解引入蛋白质组条件响应预测
- **与评测指标的闭环：**架构组件与评测模块一一对应，形成「架构-指标」可解释映射——这不是巧合，而是**根据评测指标设计的架构**
- **可验证的创新：**通过消融实验（移除残差组件 / 移除交互项）可量化每个组件的贡献

可行性分析：

- 共享组件：可用简单 MLP 从条件编码预测，对应「均值响应」，易于训练
- 残差组件：用更深网络从实体 embedding 预测偏离，规模可控
- 交互项：用注意力机制或外积建模菌株×化合物的交叉效应，参数量可控

稳定性分析：

- 残差分解的好处是**各组件分工明确**：共享组件负责稳定的基础预测（低风险），残差组件负责捕捉特异信号（有提升空间但需调参）
- 训练时如果残差组件不稳定，可先冻结共享组件，单独训练残差，再联合微调

5.4.2 创新点二：实体可迁移表征（解决零样本泛化瓶颈）

创新动机：one-hot 编码只能记住训练中见过的菌株/化合物；面对新实体，它们都是同一个「未知类别」。 `val_both`（双重未知）场景下，Baseline MLP 完全无法利用实体间的相似性。

外推原理：本任务是 OOD 任务，直接从条件生成响应，不需要输入 control。对 strain / chemical 做 embedding，**这个 embedding 就会成为外推的来源**——来了新菌株，知道它的基因组就知道 embedding，就能通过 embedding 做外推。

创新设计：引入外部知识构建可迁移表征：

- 菌株 → 基因组 → k-mer / 变异位点 / Genome LM embedding
- 化合物 → SMILES → Morgan FP / GNN / 预训练分子表征

创新性论证：

- **创新性：**多数方案只用 one-hot 编码菌株，未引入遗传背景表征；本方案在酵母蛋白质组预测中引入基因组语言模型表征，让模型能利用菌株间的序列相似性做零样本泛化
- **跨学科融合：**将计算基因组学（DNABERT-2 等）与蛋白质组学结合，是 AI4Science 的典型范式创新
- **可验证的创新：**通过 embedding shuffle / 置零消融，可证明表征信息被有效利用（shuffle 后性能应显著下降）

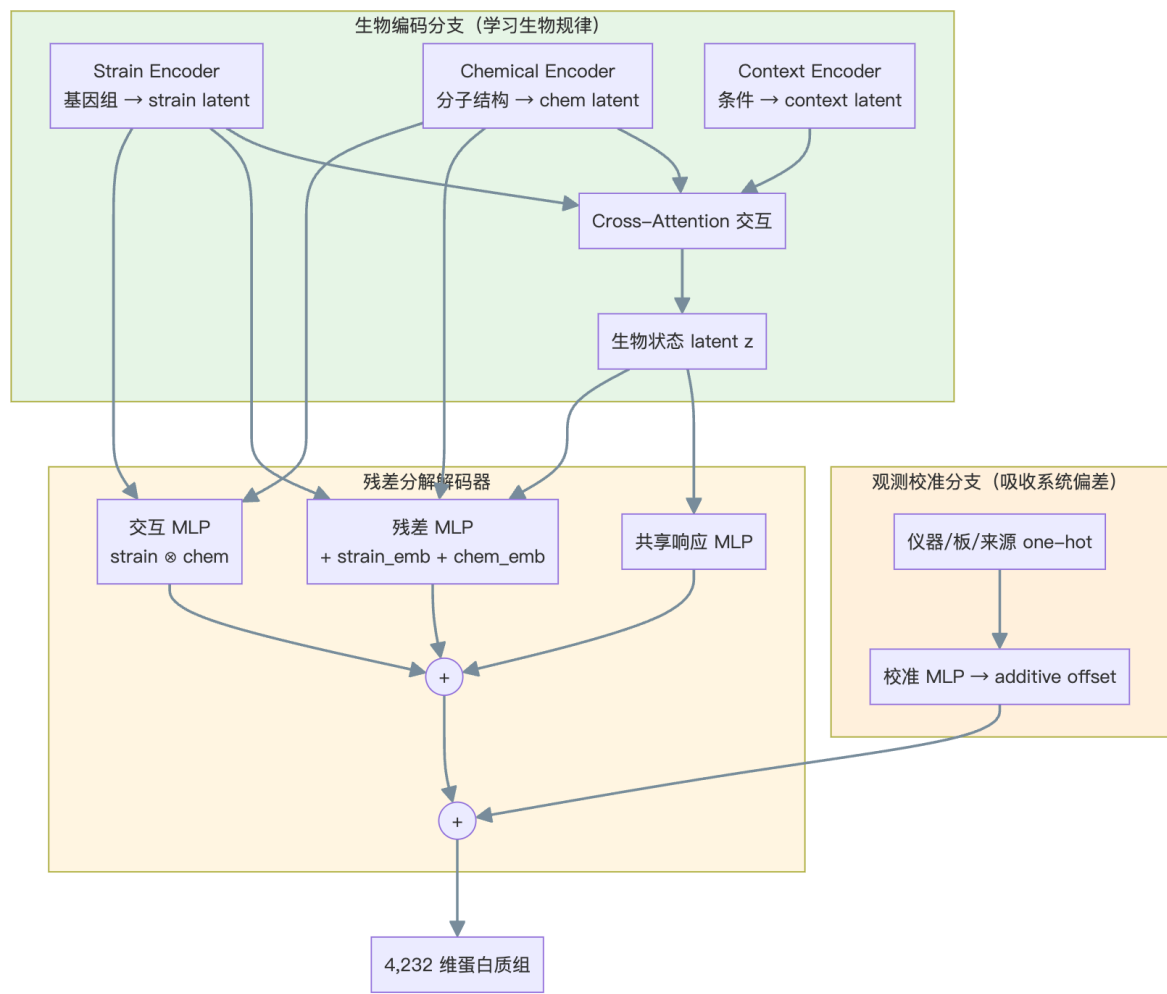
创新性提示：如果使用 genome language model embedding 并做详细的消融验证（证明 embedding 信息利用率超越 k-mer），这本身就构成一个有说服力的方法创新点。

5.4.3 创新点三：批次校准（分离生物信号与仪器偏差）

创新动机：不同的质谱仪器对不同的蛋白有不同的检测偏好——比如仪器 A 测蛋白 X 总是偏高、仪器 B 测蛋白 Y 总是偏低。如果模型把这些仪器偏差当成生物规律学进去，换一批仪器就预测不准了。

为什么会有批次效应：本赛题给的是原始质谱数据（没有标准化处理）。不同仪器、不同实验板、不同数据来源都会引入系统偏差。这些偏差不是生物现象，需要在模型中单独处理。

创新设计：将架构分为生物编码与观测校准两条独立分支：



创新性论证：

- 与常规做法的差异：多数方案会把所有元数据字段拼接后统一编码；本方案显式分离生物信号与观测过程，物理意义清晰
- 可解释性：校准分支输出为 additive offset，可解释为「该仪器/板相对基准的系统偏差」
- 稳定性贡献：观测过程独立，评测时仪器分布变化不会污染生物编码

设计要点：

- 编码器：菌株/化合物/上下文分别编码 → 拼接成生物状态向量（维度 64~256）
- 解码器：从生物状态向量回推蛋白组，引入仪器作为变量拟合批次效应
- 仪器校准分支：单独建模仪器/板/来源，输出为叠加偏差（additive offset）

5.4.4 创新点四：蛋白侧结构化先验（差异化亮点）

创新动机：大多数参赛方案只会关注条件侧（菌株/化合物），忽略蛋白侧的结构信息。引入蛋白先验可约束输出空间，降低过拟合风险。

创新设计：在蛋白侧引入多种先验：

先验类型	来源	引入方式	创新点
通路归属	GO / KEGG	通路注意力：同一通路蛋白共享注意力权重偏置	输出空间结构化
功能注释	InterPro / Pfam	蛋白侧 GCN：蛋白间按功能相似度构建图	相似蛋白约束相似响应
蛋白互作 (PPI)	STRING / BioGRID	图卷积 / 图注意力聚合	物理互作蛋白协同响应
共表达网络	训练集自计算	协方差矩阵正则化	利用训练数据内在结构

创新性论证：

- **差异化亮点：**多数方案只关注条件侧（菌株/化合物），在蛋白侧引入结构化先验是明显差异化
- **跨学科融合：**将图神经网络与蛋白互作网络结合，是结构生物学与深度学习的交叉创新
- **可验证的创新：**通过移除蛋白先验的消融实验，可量化先验信息的贡献

可行性分析：

- 通路注意力：实现简单，仅需一个通路→蛋白的 binary 矩阵，参数量小
- 蛋白侧 GCN：需要构建蛋白间图，复杂度 $O(N^2)$ 但 4,232 个蛋白可接受
- PPI 信息：可在训练集上计算蛋白间相关性，作为正则化项

5.5 新型范式创新

本节为高创新性、高难度方向，适合有相关经验的团队。即使不实现，作为方案中的「未来方向」阐述也能体现科学视野。

5.5.1 创新点五：生成式建模（捕捉分布不确定性）

创新动机：端到端回归只预测均值，无法量化预测的不确定性。虚拟实验的可靠性评估需要置信区间，而非单点预测。

创新设计：将响应建模为条件生成：

- **Conditional VAE：**编码条件 → latent distribution → 解码蛋白质组分布
- **Flow Matching：**学习状态转移的概率流，跳过高频震动，直接生成符合热力学分布的长程轨迹
- **Diffusion：**通过去噪过程生成蛋白质组分布

创新性论证：

- **与回归方法的差异：**从「预测均值」升级为「预测分布」，可量化预测不确定性

- **科学意义：**不确定性量化对虚拟实验的可靠性评估至关重要，符合 AIVC 的「enable 虚拟实验系统」愿景
- **争特别奖潜力：**如果生成式模型在隐藏机理评估中表现出对未见药物分布的合理预测，可争特别奖

可行性提示：生成式方向需要更多训练技巧（KL annealing、多尺度 loss），建议在基础模型稳定后再尝试。

5.5.2 创新点六：主动学习闭环

创新动机：与其等实验者随机补充数据，不如让模型自己说「我最不确定哪些条件，请你帮我做这些实验」。

定位说明：这是虚拟细胞的**长期目标**，**初赛不要求实现**。建议作为方案中的「未来方向」阐述，体现对 AIVC 愿景的理解，可作为科学意义（30%）的重要论述点。

创新设计：模型对所有未做实验的条件打分（基于「不确定程度」「信息量大小」等指标），推荐最值得做实验的条件组合，实验者补充数据后模型重新训练，形成循环。

创新性论证：

- **范式创新：**从「被动等数据」升级为「主动要数据」
- **实际意义：**解决虚拟细胞领域「数据太贵太少」的核心瓶颈

5.5.3 创新点组合策略

不同团队根据自身优势选择创新点组合：

团队背景	推荐创新组合	预期创新性得分
深度学习强	残差分解 + 生成式建模	15~18 / 20
生信/化学背景	实体可迁移表征 + 蛋白先验	14~17 / 20
全栈团队	残差分解 + 实体表征 + 批次校准	16~19 / 20
有 RL 经验	残差分解 + 主动学习闭环	15~18 / 20
时间有限	仅残差分解（强烈推荐起点）	10~13 / 20

5.6 训练策略提分（稳定性与准确性）

5.6.1 Loss 设计（准确性与稳定性）

Loss 组件	公式/说明	目标	权重建议
Mask-aware MSE	$\text{mean}((y_{\text{pred}} - y_{\text{true}})^2 * \text{mask})$	逐蛋白绝对丰度	1.0（主 loss）
Fold change loss	$\text{correlation}(\text{fc}_{\text{pred}}, \text{fc}_{\text{true}})$	差异表达方向	0.3~0.5
残差 loss	对残差部分加额外正则化	防止残差组件过拟合	0.1~0.2
蛋白先验正则	通路一致性等	输出空间约束	0.05~0.1

代码块

```
1  # ===== 多目标 loss 骨架 =====
2  import torch
3
4  def total_loss(y_pred, y_true, mask, fc_pred, fc_true, strain_emb, chem_emb):
5      """多目标损失: mask-aware MSE + fold change + 残差正则"""
6      # 主 loss: mask-aware MSE (逐蛋白绝对丰度)
7      loss_mse = ((y_pred - y_true) ** 2 * mask).sum() / mask.sum()
8
9      # Fold change loss: Pearson 相关系数 → 最大化即最小化 (1 - r)
10     fc_pred_flat = fc_pred[mask].flatten()
11     fc_true_flat = fc_true[mask].flatten()
12     loss_fc = 1.0 - torch.corrcoef(torch.stack([fc_pred_flat, fc_true_flat]))
13     [0, 1]
14
15     # 残差 L2 正则 (防止 embedding 过拟合)
16     loss_reg = 0.01 * (strain_emb.norm(2) + chem_emb.norm(2))
17
18     return loss_mse + 0.3 * loss_fc + loss_reg
```

稳定性说明:

- 多目标 loss 的权重需要根据各组件量级手动调节, 建议前几个 epoch 分别监控各 loss 值, 统一到相似量级
- Fold change 的计算需要确保 control 样本正确匹配, 否则 loss 方向错误

5.6.2 验证纪律 (防止虚假高分)

- **必须分桶报告:** 三类实体划分 (val_chem_only / val_strain_only / val_both) + 时间外推 (val_time), 不能只给加权总分
- **与基线对比:** 每步都要对比蛋白均值、control 基线, 证明提升来自模型而非数据固有相似性
- **消融实验:** embedding shuffle / 置零, 确认外部表征是否真起作用
- **过拟合检测:** 监控训练集与验证集 loss 差距, 若训练 loss 持续下降但验证 loss 上升, 停止训练

5.6.3 实战技巧 (借鉴高维回归比赛经验)

以下技巧来自单细胞生物学扰动预测、作用机制预测等类似比赛的获奖方案经验。

技巧 1: 后处理校准 (稳定提升 FC 类指标)

预测后用 Control 基线做偏移校准——把模型预测减去同条件 control 均值, 再加回训练集全局均值, 能稳定提升 fold change 类指标。

代码块


```

1  # 后处理校准：用 control 基线做偏移
2  control_mean = y_log2[control_samples].mean(axis=0)  # 同条件 control 均值
3  global_mean = y_log2[train_samples].mean(axis=0)      # 训练集全局均值
4  prediction_calibrated = prediction - control_mean + global_mean

```

技巧 2：多 loss 加权（比模型结构更重要）

借鉴类似比赛经验：loss 设计比模型结构对提分贡献更大。本赛题可组合多种 loss：

Loss	作用	权重建议
mask-aware MSE	基础回归	1
逐蛋白 corr loss	提升谱形状	0.1
排序 loss	提升高效应蛋白排序	0.05
Δ 一致性 loss	直接优化 FC 指标	0.2

技巧 3：分桶交叉验证（防止过拟合到特定 split）

借鉴类似比赛经验：分桶验证比随机划分更接近真实泛化。本赛题已有官方分桶，但内部研究可做更细的 k-fold：

代码块

```

1  # 按 (strain, chemical) 组合做 k-fold, 确保同组不跨 fold
2  from sklearn.model_selection import GroupKFold
3  groups = metadata['strain'] + '_' + metadata['chemical']
4  gkf = GroupKFold(n_splits=5)
5  for train_idx, val_idx in gkf.split(X, y, groups):
6      # 每个 fold 的验证集包含未见组合
7      ...

```

技巧 4：蛋白间相关性建模（类似比赛核心技巧）

蛋白间存在功能相关性（同一通路的蛋白协同变化）。可在 loss 中加入蛋白间相关性约束：

代码块

```

1  # 蛋白相关性 loss: 预测的蛋白间相关性矩阵应接近真实
2  pred_corr = torch.corrcoef(pred.T)  # 预测的蛋白间相关
3  true_corr = torch.corrcoef(y_true.T)  # 真实的蛋白间相关
4  corr_loss = ((pred_corr - true_corr) ** 2).mean()

```

技巧 5：TTA（测试时增强）

对同一测试样本，用不同随机种子做多次预测取平均，降低预测方差。适合 FC 类指标。

5.6.4 防泄漏（准确性底线）

泄漏源	后果	对策
缺失率过滤借用验证集	过滤掉的蛋白在验证集上也有缺失，间接利用验证信息	缺失率仅从 训练行 计算
标准化统计量借用验证/测试	均值/标准差含验证信息，间接泄漏	标准化统计量仅从训练集估计
读取测试蛋白质组真值	直接泄漏	流程中不加载测试真值文件
正交划分信息混用	验证集菌株/化合物信息混入训练	严格遵守 split_final 字段划分

5.7 科学意义与创新性阐述（占 50%，差距拉开处）

创新性的具体技术方案已在 5.4（架构创新）和 5.5（范式创新）详述。本节聚焦于**如何在方案文档中阐述**科学意义与创新性，让评审快速理解你的贡献。

5.7.1 科学意义（30%）阐述策略

（1）生物学解读 —— 让评审看到「你懂生物学」

对预测结果做下游分析，而非只报告指标：

分析	方法	向评审说明什么
通路富集	对预测的 fold change 做 GO/KEGG 富集，检查是否富集到已知的药物靶向通路	模型抓住了真实生物学信号
关键蛋白验证	选取 fold change 最大的 10 个蛋白，查文献确认是否与对应化合物机理相关	预测有生物学依据
菌株特异性分析	同一化合物在不同菌株上的响应差异，是否与菌株基因型差异一致	模型学到了菌株-化合物交互
时间序列分析	预测的蛋白强度随时间变化是否符合生物学预期（如应激→恢复）	模型学到了动态规律

（2）外部数据虚拟实验 —— 评分最高杠杆

- 在赛题外部的未见药物数据上做虚拟实验，预测其影响
- 这是「隐藏机理评估」的设计逻辑，表现好可争**特别奖**（指标低但机理对的情况酌情考虑）
- 可行性：挑选 3~5 个在训练集中未见、但在文献中有明确蛋白质组响应的化合物，用模型预测并与文献结果对比

（3）科学问题阐述 —— 清晰说明三个「为什么」

- 为什么这个问题值得做？（AIVC 是计算生物学圣杯，酵母是最好的起点）
- 为什么你的方法能解决它？（残差分解对齐评测、外部表征驱动泛化）

- 为什么你的结果超越了已有工作？（在蛋白质组 + 多菌株 + 多条件上首次系统验证残差分解 + 实体表征的有效性）

5.7.2 方法创新性（20%） 阐述策略

创新点总览（对应 5.4 / 5.5 详述的六大创新点）：

创新点	创新强度	落地难度	对应章节	建议
残差分解架构	中	低	5.4.1	强烈推荐起点 ：首次引入蛋白质组预测，天然对齐评测指标
实体可迁移表征	高	中	5.4.2	推荐：首次用 Genome LM 做菌株跨背景泛化
批次校准分支	中	中	5.4.3	推荐：分离生物信号与观测过程
蛋白侧结构化先验	高	中	5.4.4	推荐：差异化亮点，多数方案只关注条件侧
生成式建模	高	高	5.5.1	加分项：适合有生成模型经验的团队
主动学习闭环	极高	高	5.5.2	作为「未来方向」阐述，不要求实现

创新描述模板（使评审快速理解贡献）：

代码块

1

1. 领域现状：在蛋白质组条件响应预测中，现有方法 [简述不足]

2

2. 我们的贡献：首次提出 [创新点]，其优势在于 [对齐评测/解决瓶颈]

3

3. 验证：通过 [消融实验]，证明 [创新点] 带来 [量化提升]

4

4. 边界：该方法的局限性在于 [诚实说明]，未来可通过 [改进方向] 解决

5.8 复现与开源加分（5% 保底 + 复赛门槛）

可复现是门槛：复现不出来则不参与计分。开源贡献虽仅 5%，但复赛/决赛要求可运行代码仓库，是晋级硬门槛。

5.8.1 复现要求（复赛起强制）

要求	具体内容
运行入口	单命令启动（如 python main.py --config config.yaml 或 bash run.sh）
依赖安装	requirements.txt 或 environment.yml，含精确版本号
配置方法	所有超参数集中在一个 config 文件，注释清晰
随机种子	固定所有随机种子（Python、numpy、PyTorch），确保结果可复现
复现说明	README 中说明运行步骤、预期输出、预期运行时间
依赖披露	开源依赖、商业服务、外部数据来源、许可证、关键版本
商业 API/闭源模型	披露调用环节、费用假设、权限范围、可替代性、对可复现性的影响

5.8.2 开源计划与边界

- 开源代码、模型权重、数据处理管线
- 明确开源边界（哪些公开、哪些因数据合规不公开）
- 提供可被社区接续使用的接口与文档
- **建议：**初赛即开始用 Docker 固定环境、用 MLflow 记录实验，复赛直接产出可复现仓库

5.9 提分路线图（由易到难，每步可独立验证）



5.9.1 每阶段可行性自检表

阶段	通过标准	未通过时的回退方案	封顶风险
Lv0（Baseline）	基线指标与赛题报告一致（±5%）	排查对齐/过滤/尺度，无回退（必须过）	无
Lv1（Baseline）	MLP 逐蛋白 R² 中位数 > 0，超过 Control 基线	回退到 Matched Control 基线	可能过拟合到训练分布
Lv2（提分）	val_both 逐蛋白 R² 相比 Lv1 提升 ≥10%	回退到 one-hot，检查外部表征匹配正确性	外部表征质量不足
Lv3（提分）	fold change 残差指标相比 Lv2 提升 ≥5%	逐一消融交互/先验/校准，仅保留有效模块	交互模块引入过拟合
Lv4（提分）	通路富集分析有显著生物学意义	仅做指标分析，不做过度解读	外部数据不匹配

5.9.2 提分避坑清单

坑	影响维度	后果	对策
NA 填 0	准确性	数据分布破坏，模型学到虚假信号	mask 矩阵，仅有值处算梯度
z-score 直接提交	准确性	评测端无法统一比较	输出前还原到 log2
用验证/测试算缺失率	准确性	信息泄漏，验证指标虚高	仅用训练行
只看 Global R ²	准确性	自欺欺人（均值都能 0.87+）	逐蛋白 R ² 、fold change、残差、分场景
名称↔SMILES 错配	准确性	化合物表征完全失效	保留 CID/SMILES/来源/人工复核
embedding 无消融	创新性	无法证明表征有效，评审质疑	shuffle/置零对比
不分桶验证	稳定性	被大样本场景掩盖，泛化评估失真	三类实体 + 时间分别报告
不可复现	可行性	一票否决，不参与计分	固定种子、Docker、复现说明
观测过程混入生物编码	稳定性	学到仪器偏好，评测时泛化崩溃	仪器校准用独立分支
残差组件发散	稳定性	训练不收敛，loss 震荡	先冻结共享组件，单独训练残差
Baseline 引入复杂模块	可行性	不确定性放大，难以归因问题	Baseline 只用 MLP，复杂模块留到提分阶段

六、初赛方案文档撰写（从教程到提交）

本章将教程内容映射为初赛方案文档的标准结构，并提供撰写辅助 prompt 与四性自查清单。

6.1 提交要求与合规说明

初赛提交物：方案说明文档 + 技术路线概述（不强制提交代码，可附初步实验或可行性验证）。

赛事要求：初赛材料应说明所选方向、科学问题理解、技术方案与预期方法路线；可附初步实验或可行性验证。重点考察核心思考，不以篇幅长短为标准。

关于 AI 辅助的边界：

可以使用 AI 进行辅助（如梳理素材、检查框架、润色语言），但**必须有自己的原创性思考，不能完全按照 AI 回答的结果提交**。评委重点看的是你对赛题的独立理解与判断——哪些是你跑通后得到的经验、哪些是你方案的取舍理由、哪些是你结果的归因分析。这些核心思考必须来自你自己，AI 无法替代。

⚠️ 自检标准：提交前对每一段文字问自己「这段话我能在答辩时解释清楚吗？我能说出为什么这么写吗？」如果答不上来，说明这段缺乏你的思考，需要改写或补充自己的理解。直接照搬 AI 输出或教程原文，答辩时无法自治。

6.2 方案文档结构与教程内容映射

初赛方案文档严格按官方大纲四章结构组织，与本教程章节的对应关系如下。**撰写时用自己的语言重组，不要直接复制教程原文**（教程是教你怎么做，提交文档是讲你要怎么做）；文档呈现的是**完整最**

终方案（含创新模块），不是 Baseline 分阶段版本；关键信息用表格/流程图呈现，决策理由用段落陈述。

文档章节	应说明内容	教程素材来源	撰写要点
一、项目概述			
1.1 项目名称	项目名称	1.1	一句话，体现「条件响应细胞」
1.2 参赛方向	报名时选定的算法赛题方向	1.2 + 3.1.2a	简明说明方向与榜单选择，即可，不展开评审机制
1.3 方案概述	所选方向、科学问题、技术方案与预期方法路线	1.3 + 3.1.6 + 5.4	一段话讲清完整方案全貌 Mermaid 总体架构图（最后阶段）
二、科学问题理解			
2.1 科学问题与研究对象	对所选方向科学问题的理解	2.1	段落陈述 + 一张「量化难点」
2.2 科学意义	该方法或结果对所选科学领域的意义	2.2 + 2.1.2 概念演进脉络	应用价值 + 方法价值 + 与业界（评审口径「是否超出业界」）
三、技术方案与预期方法路线			
3.1 技术方案	针对所选科学问题的技术方案	3.1 全章 + 5.3 + 5.4 + 3.1.4a + 5.9.2	从创新方向池自主选 3~5 个，完成完整方案；呈现方案架构图、工程表、稳定性表、创新点列表、风险表；创新点按「异常设计→对应评测→消融实验」
3.2 预期方法路线	预期采用的方法路线	3.2 + 5.9	一张 Mermaid 研发路线图 进阶表（各阶段是构建完整方案的步骤）
3.3 数据来源、依赖工具与运行流程	数据来源、依赖工具与运行流程	3.3 （结构完全对应）	数据来源表 + 依赖工具表 + Mermaid 运行流程图
四、阶段性实验结果或可行性验证			
4.1 阶段性实验或可行性验证	已完成的初步实验或可行性验证	4.1 + 4.3	一张「可行性论证表」（含可行性） + 一张「检查点清单」（通过状态）
4.2 当前结果	阶段性实验结果或可行性验证结果	4.2	一张官方基线对标表（标准） + 一张诊断归因表；注明数据差异

撰写顺序建议：先写三、四章（技术方案+可行性验证，评审最看重），再写第二章（科学问题），最后写第一章（概述）。

附录 A：术语速查表

遇到不认识的术语？先来这里查。

生物学术语

术语	含义	在赛题中的角色
蛋白质组（Proteome）	细胞中所有蛋白质的集合	预测目标（4,232 维向量）
酿酒酵母（ <i>S. cerevisiae</i> ）	一种真核微生物，模式生物	研究对象（6 株）
菌株（Strain）	同一物种的遗传变体	输入条件之一（6 类）
扰动（Perturbation）	改变细胞状态的处理（化学/遗传）	化合物处理即为扰动
Fold change	处理组 vs 对照组的倍数变化	评测指标之一
通路（Pathway）	一组协同完成生物功能的蛋白	蛋白先验信息来源
质谱（Mass spec）	测量蛋白质丰度的仪器	数据产生方式，引入批次效应

技术术语

术语	含义	何时用到
OOD（Out-of-Distribution）	分布外泛化，预测训练时未见过的实体	本赛题核心挑战
mask-aware	用 mask 矩阵屏蔽缺失值梯度	数据预处理 + loss 计算
log2 尺度	对原始强度取 log2	评测统一尺度
one-hot 编码	类别 → 二值向量	Baseline 条件编码
Embedding	类别 → 可学习稠密向量	Baseline/提分条件编码
残差分解	预测 = 共享响应 + 特异残差	提分架构创新点
Cross-attention	两个模态间的注意力交互	提分交互模块
SMILES	分子的线性表示字符串	化合物表征
Morgan fingerprint	分子结构的位向量表示	化合物特征起点
1,011 project	1,011 株酵母全基因组测序资源	菌株表征来源
正交划分	按实体（菌株/化合物）划分验证集	验证集设计
分桶验证	分场景分别报告指标	验证纪律

评测指标术语

术语	含义	权重	区分度
原始 FC（匹配对照）	$\Delta = \hat{y}_{\text{treat}} - y_{\text{control}}$ ，算 $\text{PCC}(\Delta_{\text{pred}}, \Delta_{\text{true}})$	25%	高（权重最高）
绝对保真度	逐样本 corr/R^2 + 逐蛋白 corr/R^2	20%	中
上下文均值残差	减同上下文训练药物 Δ 均值后的 PCC	20%	高（化合物特异）
药物均值残差	减同药物训练上下文 Δ 均值后的 PCC	20%	高（菌株调制）
双重未知/时间外推	FC + 绝对保真度，不用残差	10%	高
DEP 检出	$ \Delta > 1$ 蛋白的方向准确率/F1	5%	低
Global R^2	全局 R^2 ，所有样本一起算	非独立模块	极低（均值 0.87+）
Matched Control 基线	同条件对照的均值预测	—	强基线（需超越）
Δ （fold change）	$\log_2$ 空间： $\hat{y}_{\text{treat}} - y_{\text{control}}$	—	65% 分数的核心